projet_ML

Adélie, Yelli, Paulina

2026-03-10

PROJET DU COURS DE COMPLEMENT DE MATHEMATIQUES 2

Introduction

Contexte

Dans ce projet, nous travaillons sur un jeu de données regroupant des informations détaillées sur les employés d'une entreprise, incluant leurs caractéristiques démographiques (âge, genre, ville), leur parcours académique (niveau d'études), leur historique professionnel (année d'entrée, expérience dans le domaine), ainsi que des éléments liés à leur situation de travail (niveau de rémunération, périodes éventuelles de "bench"). Les données ont été anonymisées afin de préserver la confidentialité tout en permettant une analyse approfondie des dynamiques internes de la main-d'œuvre. Ce type de dataset est couramment utilisé dans les études de ressources humaines, notamment pour analyser les facteurs associés au turnover, à la rétention, à la diversité ou encore à la structure salariale.

L'idée générale est d'utiliser ces informations pour mieux comprendre le comportement des employés et plus précisément pour savoir **quels facteurs peuvent expliquer le départ d'un employé**.

Objectif de l'étude

L'objectif central de cette analyse est de décrypter les facteurs d'influence qui poussent un collaborateur à quitter l'entreprise afin de prédire le risque de turnover.

Cette démarche s'articule autour des axes suivants :

- Caractériser les profils des employés en distinguant les traits saillants de ceux qui restent face à ceux qui partent.
- Isoler les variables explicatives (démographiques, salariales et professionnelles) les plus pertinentes pour modéliser le départ.
- Structurer et traiter les données pour permettre l'entraînement d'un modèle de classification efficace sur la variable `LeaveOrNot`.

Source des données

Le jeu de données utilisé dans ce projet est disponible à l'adresse suivante :

https://www.kaggle.com/datasets/tawfikelmetwally/employee-dataset

1 - Description des variables :

Le dataset contient 9 variables, réparties en variables qualitatives, quantitatives et une variable cible.

1.1 - Variables qualitatives :

Ces variables représentent les catégories et les attributs non numériques des employés :

- **Education** : Niveau d'études atteint (Bachelors, Masters, PHD).
- **City** : Ville d'affectation de l'employé (Bangalore, Pune, New Delhi).
- **Gender** : Genre de l'employé (Male, Female).
- **EverBenched** : indique si l'employé a déjà été sans mission temporairement (Yes/No).

1.2 - Variables quantitatives :

Ces variables mesurent des données numériques et temporelles :

- **JoiningYear** : Année d'intégration au sein de l'entreprise.
- **PaymentTier** : Segmentation de la rémunération (échelonnée de 1 pour le niveau le plus faible à 3 pour le plus élevé).
- **Age** : Âge actuel de l'employé.
- **ExperienceInCurrentDomain** : Nombre d'années d'expérience acquises dans le domaine d'activité actuel.

1.3 - Variable cible

La variable cible est la variable que l'on cherche à prédire. - `LeaveOrNot` : - `0` : l'employé reste dans l'entreprise - `1` : l'employé quitte l'entreprise

1.5 - Chargement et découverte des données

Dans un premier temps, nous importons la base de données puis nous observons sa structure générale.

```
data_project=read.csv("Employee.csv")
```

Déterminons le nombre de lignes et de colonnes de la base de données et un aperçu du jeu de données.

```
dim(data_project)      #dimension de la base de données
```

```
## [1] 4653    9
```

```
head(data_project)
```

```
##   Education JoiningYear      City PaymentTier Age Gender EverBenched
## 1 Bachelors        2017 Bangalore           3  34   Male          No
## 2 Bachelors        2013      Pune           1  28 Female          No
## 3 Bachelors        2014 New Delhi           3  38 Female          No
## 4   Masters        2016 Bangalore           3  27   Male          No
## 5   Masters        2017      Pune           3  24   Male         Yes
## 6 Bachelors        2016 Bangalore           3  22   Male          No
##   ExperienceInCurrentDomain LeaveOrNot
## 1                         0          0
## 2                         3          1
## 3                         2          0
## 4                         5          1
## 5                         2          1
## 6                         0          0
```

2 - Analyse unavriée

COMMENTAIRE... Ce que nous cherchons à prédire est le fait de savoir si un individu va quitter ou non l'entreprise. Nous allons donc travailler avec la variable LeaveOrNot qui est une variable binaire 0 si l'employé reste et 1 si l'employé part

```r
target = factor(data_project$LeaveOrNot, labels = c ("Rester", "Partir"))
data_project$LeaveOrNot = as.factor(data_project$LeaveOrNot)
summary(data_project$LeaveOrNot)
```

```
##    0    1
## 3053 1600
```

Au vu de nos données, nous considérons que toutes les variables sont toutes utiles pour notre étude, nous n'allons donc pas supprimer ou fusionner des variables de notre jeu de données.

2.1 - Répartition

```r
quantitatives_vars= data.frame(JoiningYear= data_project$JoiningYear ,
                               PaymentTier=data_project$PaymentTier,
                               Age=data_project$Age,
                               Experience=data_project$ExperienceInCurrentDomain)

qualitatives_vars= data.frame(Education=data_project$Education,
                              City= data_project$City,
                              Sex=data_project$Gender,
                              Benched=data_project$EverBenched)
```

2.2. Analyse statististique

2.2.1. Variables quantitatives

Effectuer l'analyse statistique descriptive de chacune des variables quantitatives incluses dans `quantitative_vars`

```r
summary(quantitatives_vars)
```

```
##   JoiningYear    PaymentTier         Age          Experience
##  Min.   :2012   Min.   :1.000   Min.   :22.00   Min.   :0.000
##  1st Qu.:2013   1st Qu.:3.000   1st Qu.:26.00   1st Qu.:2.000
##  Median :2015   Median :3.000   Median :28.00   Median :3.000
##  Mean   :2015   Mean   :2.698   Mean   :29.39   Mean   :2.906
##  3rd Qu.:2017   3rd Qu.:3.000   3rd Qu.:32.00   3rd Qu.:4.000
##  Max.   :2018   Max.   :3.000   Max.   :41.00   Max.   :7.000
```

COMMENTAIRE

2.2.2. Variables qualitatives

Pour chacune des variables qualitatives incluses dans `qualitative_vars`

```r
summary(qualitatives_vars)
```

```
##   Education             City               Sex              Benched
##  Length:4653        Length:4653        Length:4653        Length:4653
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
```

COMMENTAIRE

2.2.3. Target

Indiquer le nombre d'observations tombées dans chacune des deux catégories de la réponse

```r
summary(target)
```

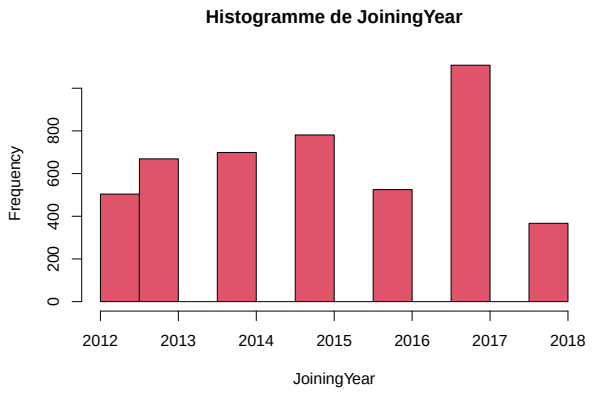
```
## Rester Partir
##   3053   1600
```

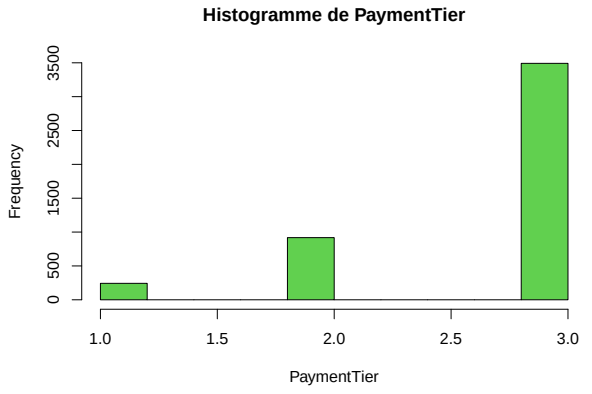
COMMENTAIRE

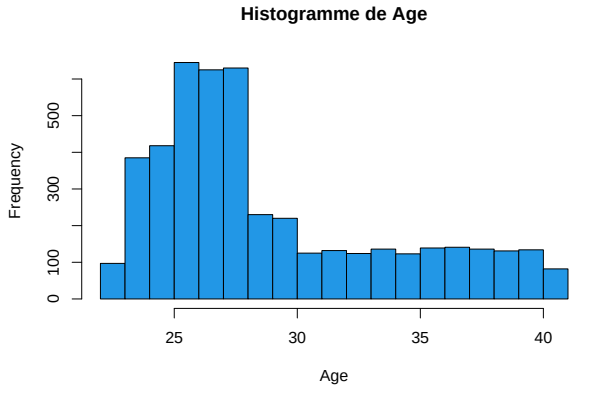
2.3. Analyse graphique

2.3.1 - Les Variables quantitatives

```r
for(i in 1:ncol(quantitatives_vars)){
  main_i = paste0("Histogramme de ", colnames(quantitatives_vars)[i])
  hist(quantitatives_vars[,i], col = i+1, main = main_i,
       xlab = colnames(quantitatives_vars)[i])
}
```


Histogramme de JoiningYear
Frequency
0
200
400
600
800
2012
2013
2014
2015
2016
2017
2018
JoiningYear

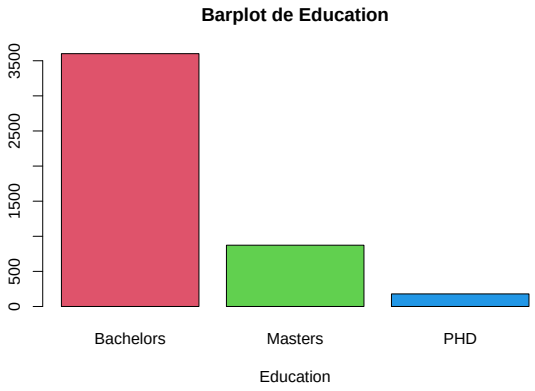

Histogramme de PaymentTier
Frequency
0
500
1500
2500
3500
1.0
1.5
2.0
2.5
3.0
PaymentTier

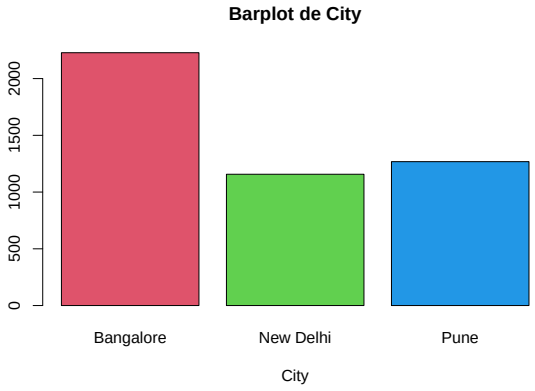

Histogramme de Age
Frequency
0
100
300
500
25
30
35
40
Age

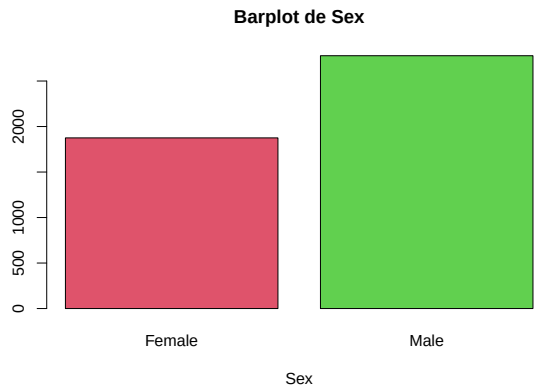
COMMENTAIRE

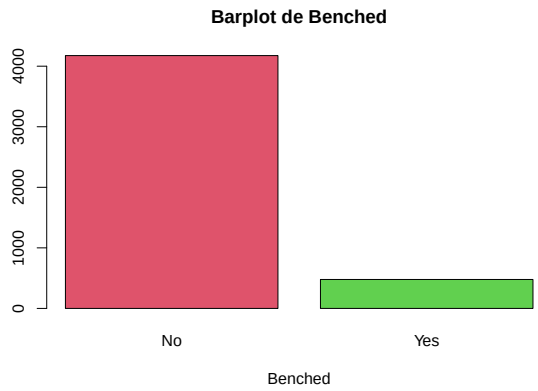
2.3.2 - Les Variables qualitatives

```
for(i in 1:ncol(qualitatives_vars)){
  main_i = paste0("Barplot de ", colnames(qualitatives_vars)[i])
  barplot(table(qualitatives_vars[,i]), col = 2:10, main = main_i,
          xlab = colnames(qualitatives_vars)[i])
}
```









COMMENTAIRE

#3 - Analyse multivariée

Il est important d'étudier la relation entre les différentes variables explicatives car certaines peuvent parfois être redondantes. De plus, nous allons vérifier qu'il n'y est pas une forte colinéarité entre variables pouvant poser problème en rendant les modèles moins fiables ou plus difficiles à interpréter.

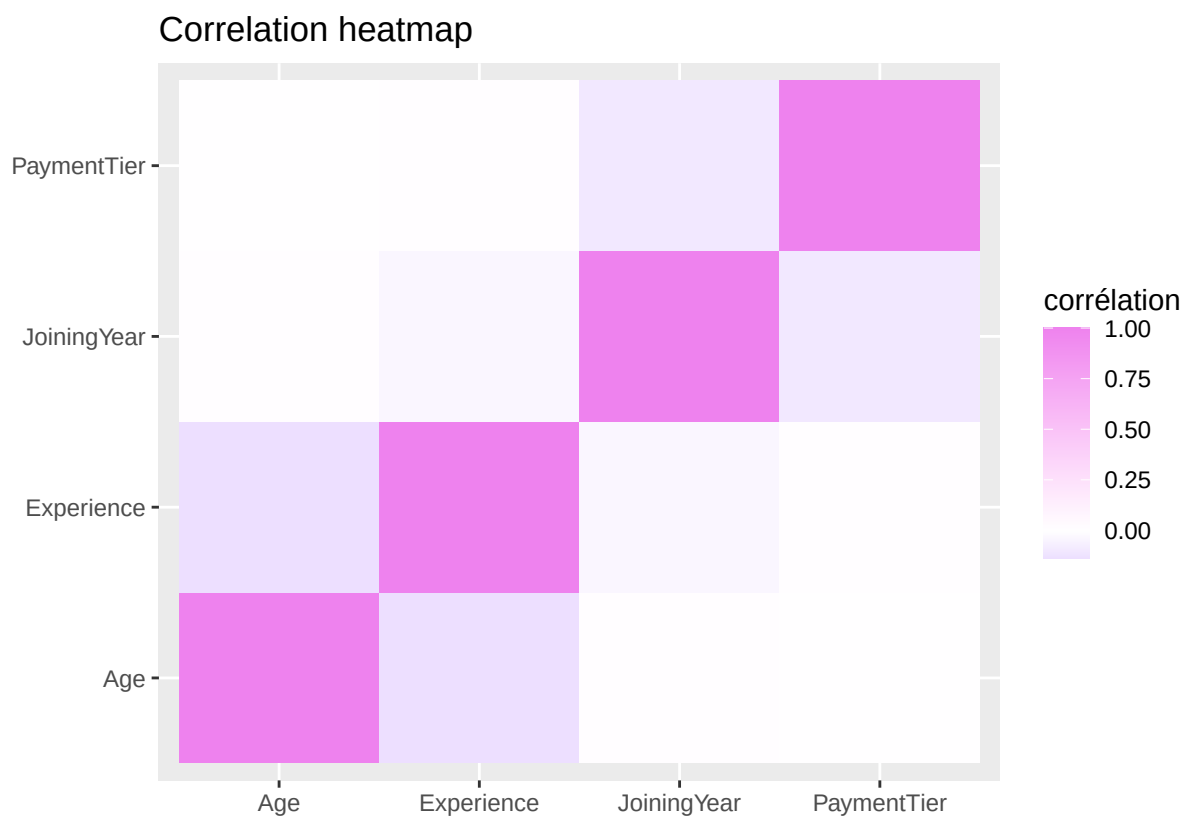
3.1 - Matrice corrélation

Dans un premier temps, on calcule la corrélation de chacune des paires de variables quantitatives puis on représente graphiquement leurs valeurs.

```
library(reshape2)
library(ggplot2)
quantitatives_vars = data.frame(
  Age = data_project$Age,
  Experience = data_project$ExperienceInCurrentDomain,
  JoiningYear = data_project$JoiningYear,
  PaymentTier = data_project$PaymentTier
)

# matrice de corrélation
cor_mat = cor(quantitatives_vars, use = "complete.obs")
cor_mat
```

```
##                     Age  Experience JoiningYear PaymentTier
## Age          1.00000000 -0.13464285  0.01316529  0.00763090
## Experience  -0.13464285  1.00000000 -0.03652462  0.01831432
## JoiningYear  0.01316529 -0.03652462  1.00000000 -0.09607821
## PaymentTier  0.00763090  0.01831432 -0.09607821  1.00000000
```

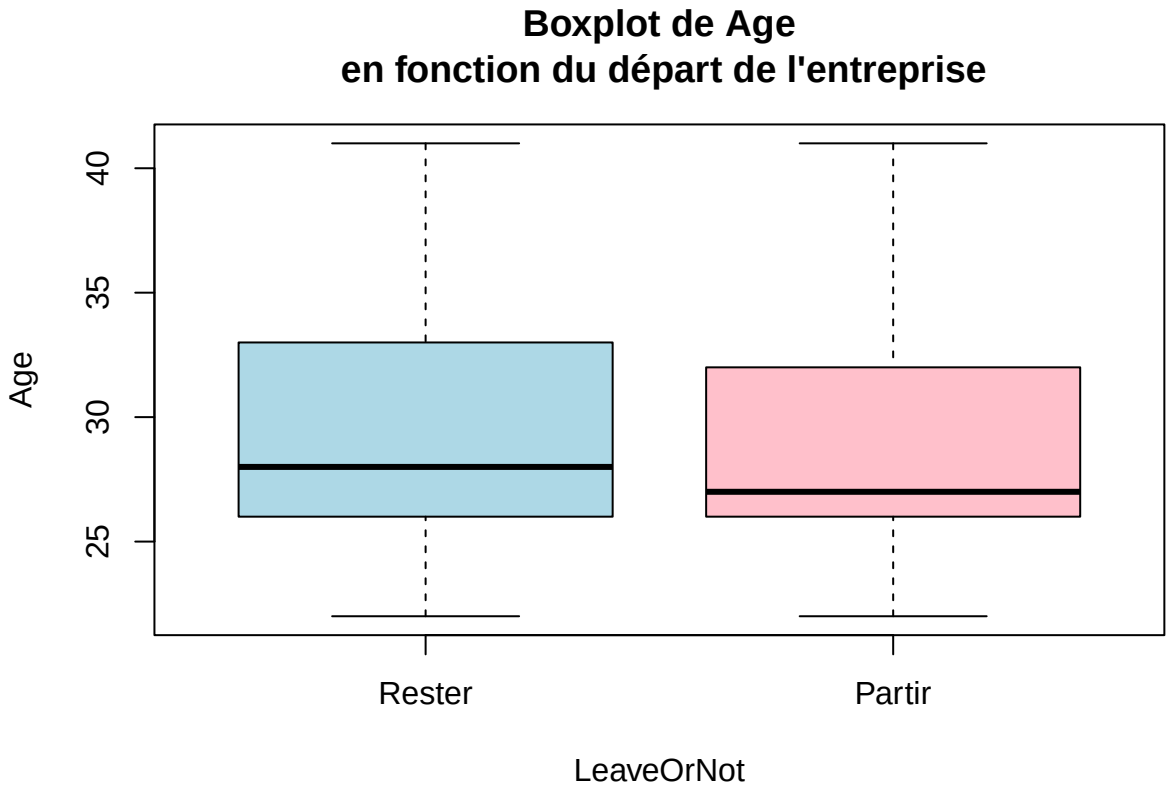
```r
# heatmap
melted_cor_mat = melt(cor_mat)

ggplot(data = melted_cor_mat, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue", mid = "white", high = "violet") +
  labs(title = "Correlation heatmap", x = "", y = "", fill = "corrélation")
```

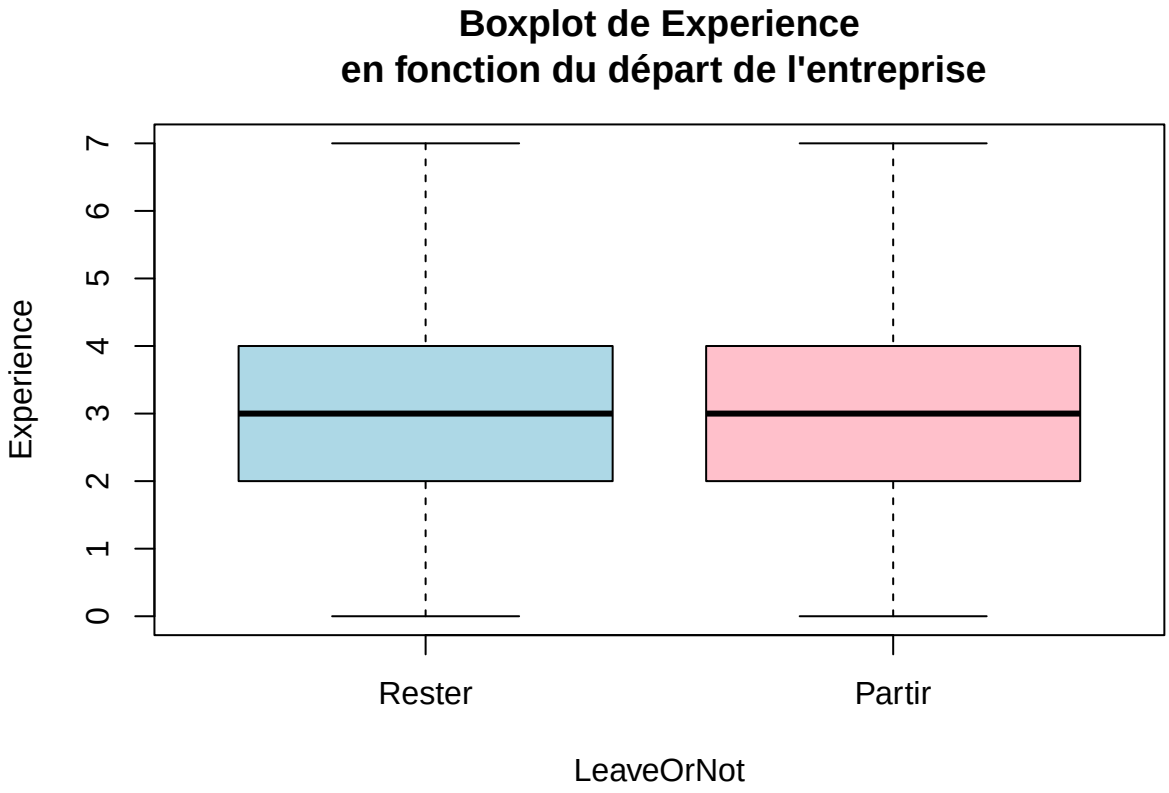


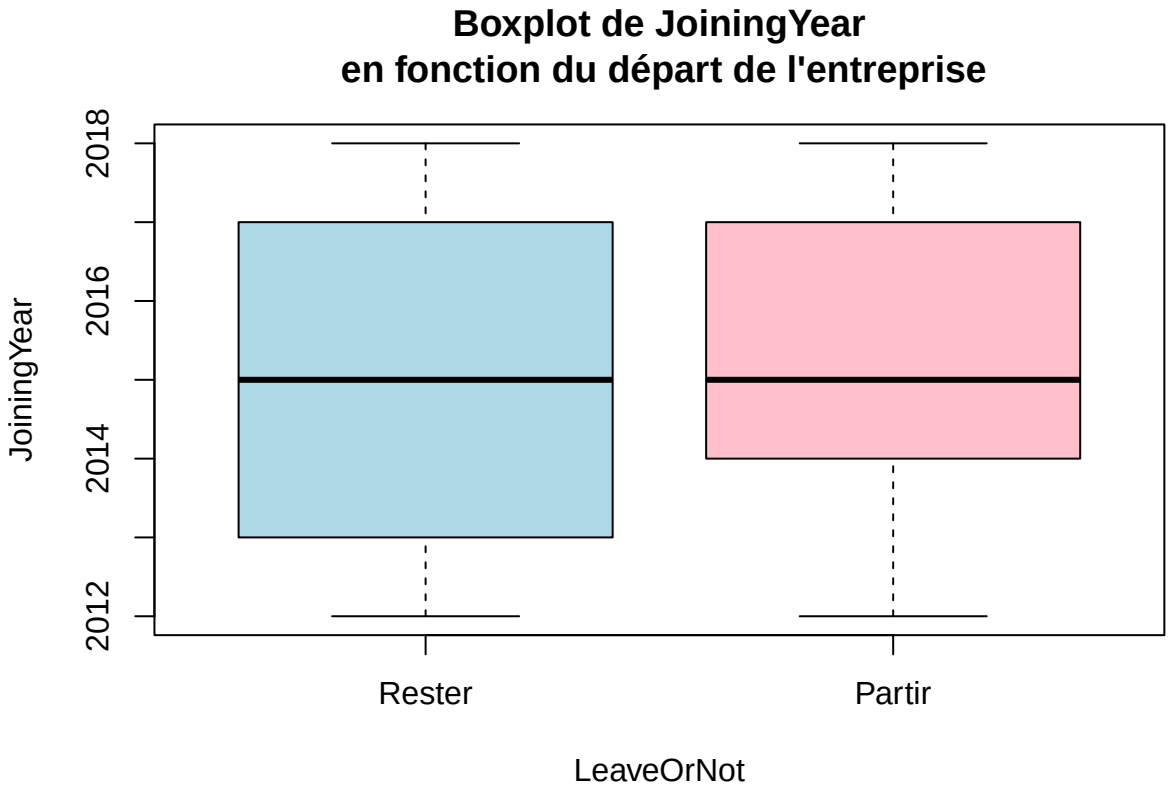
On ne remarque aucune forte corrélation entre les variables quantitatives (inférieur à 0.1 pour toutes), celle-ci étant légèrement plus haute entre l'âge et l'expérience, mais restant tout de même assez faible, ce qui est bon signe.

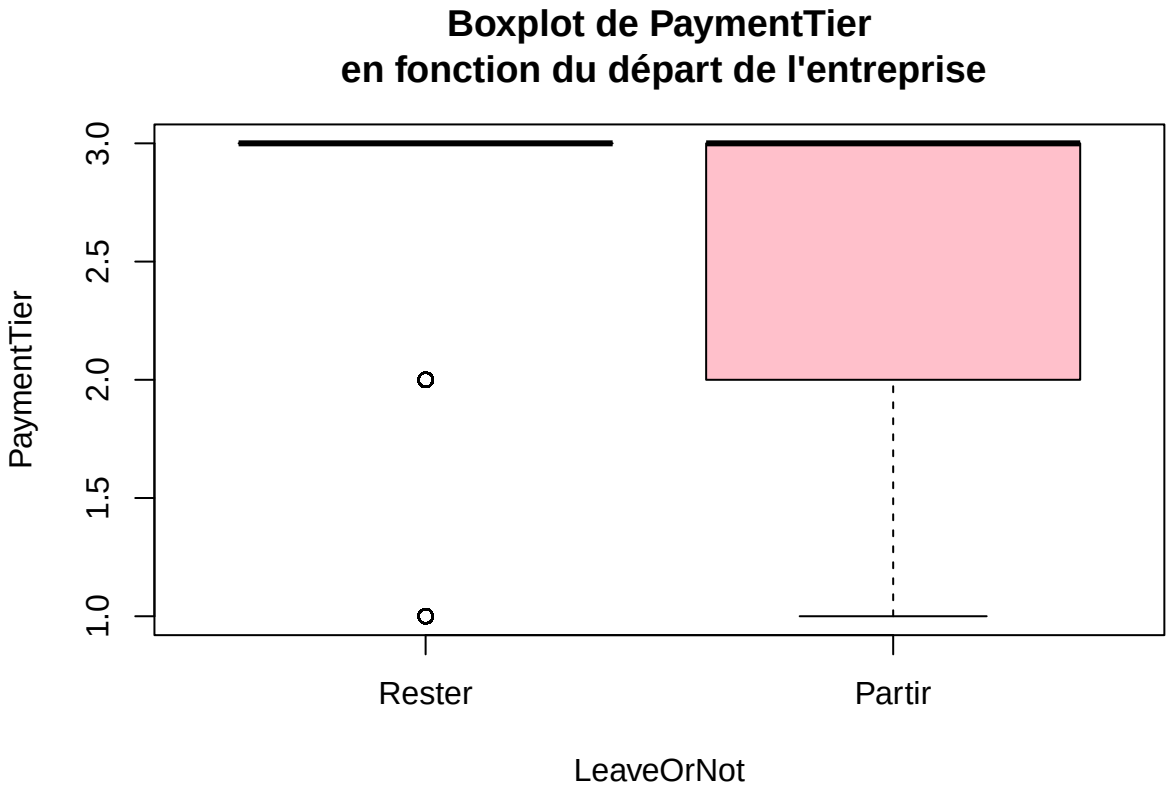
3.2 Variables quantitatives et target

On représente graphiquement la relation entre chacune des variables quantitatives incluses dans `quantitative_vars` et la target


Boxplot de Age
en fonction du départ de l'entreprise
Age
25
30
35
40
Rester
Partir
LeaveOrNot


Boxplot de Experience
en fonction du départ de l'entreprise
Experience
0
1
2
3
4
5
6
7
Rester
Partir
LeaveOrNot


Boxplot de JoiningYear
en fonction du départ de l'entreprise
JoiningYear
2012
2014
2016
2018
Rester
Partir
LeaveOrNot



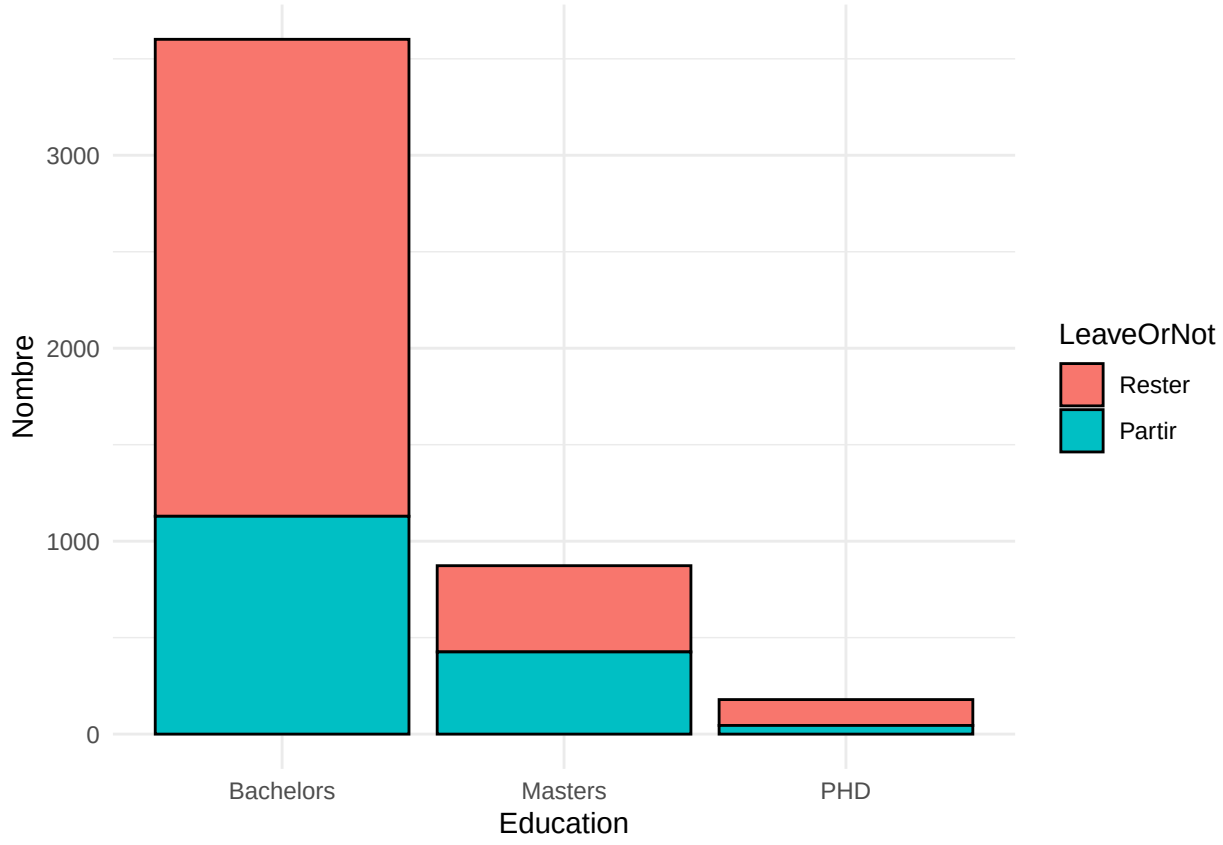
On remarque que l'âge, l'expérience et l'année ou l'employée rejoins l'entreprise ne semble pas avoir d'impacte car ils ont des distributions très similaires. Cependant, le niveau de paiement de l'employée a un fort impacte : la plupart des employés restant dans l'entreprise, voir quasiment tous, sont de la catégorie 3.

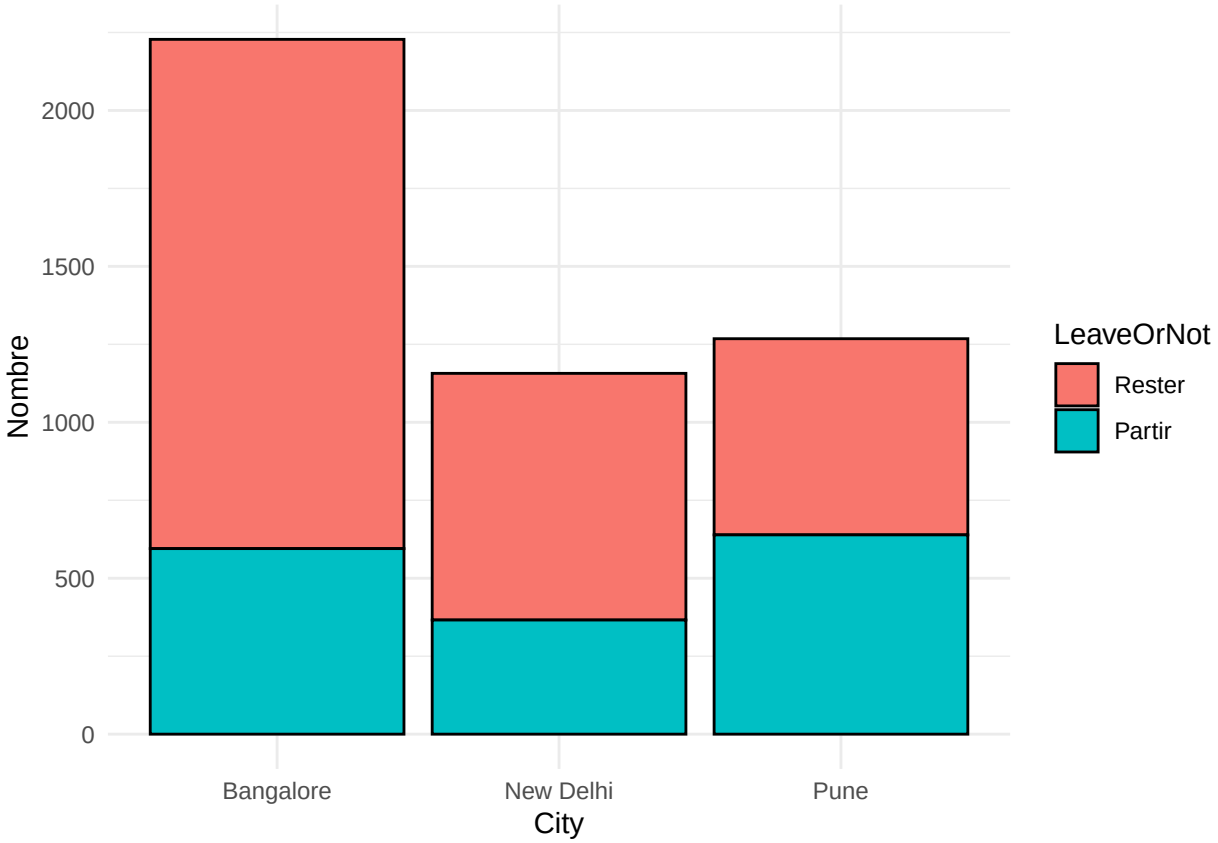
3.3 - Variables qualitatives et target

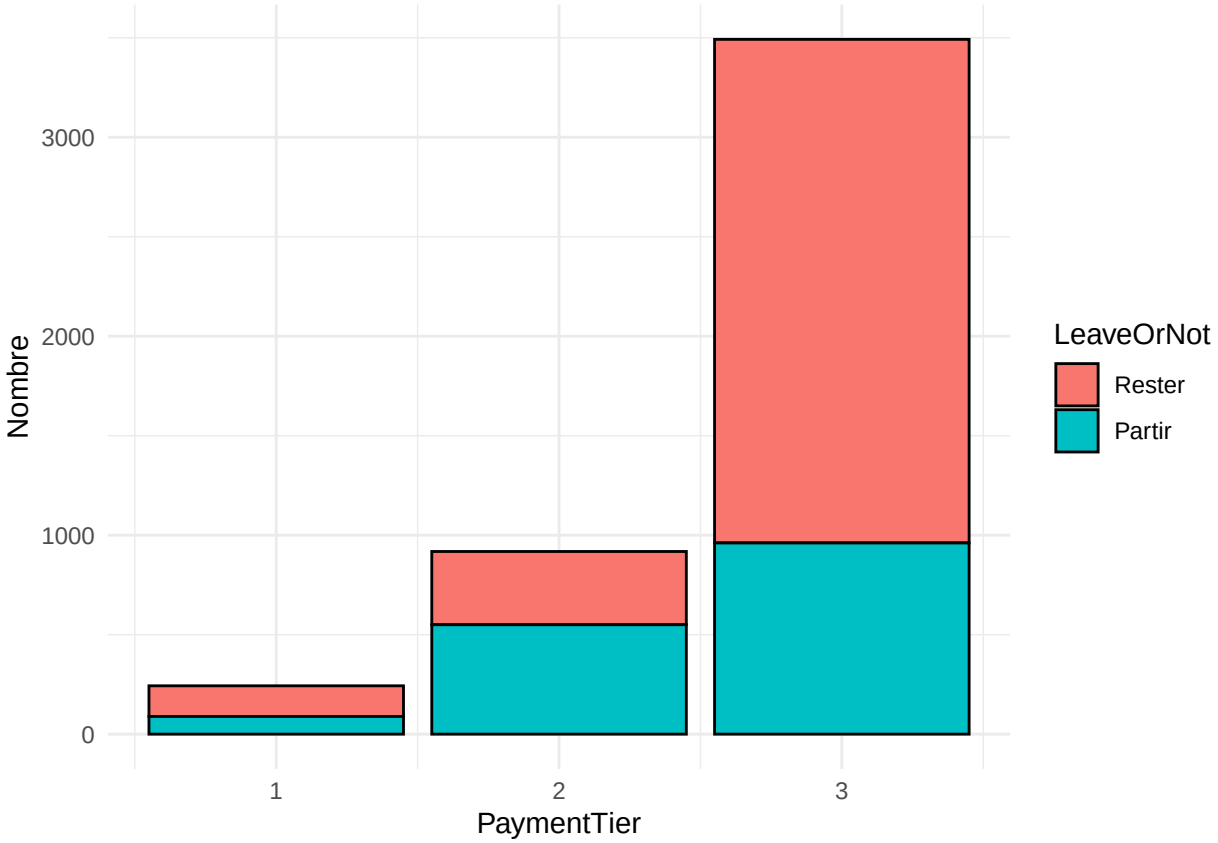
Observons maintenant le nombre d'individu dans chacune de nos différentes catégorie dans `qualitative_vars` :

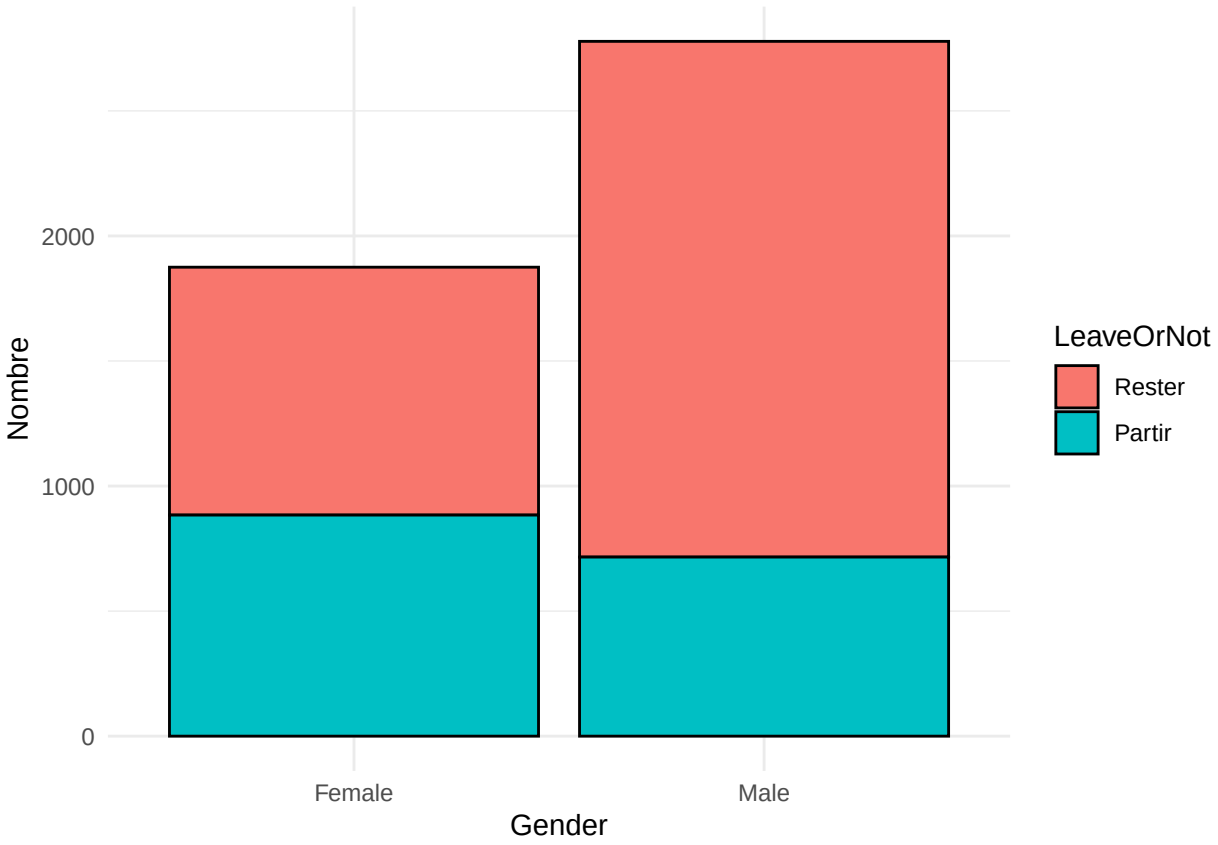
```
library(ggplot2)
library(reshape2)

qualitatives_vars = data.frame(Education = as.factor(data_project$Education), City = as.factor(data_pro

for(i in 1:ncol(qualitatives_vars)){

  var_target_counts = melt(table(qualitatives_vars[,i], target))

  xlab_i = colnames(qualitatives_vars)[i]

  print(
    ggplot(data = var_target_counts,
           aes(x = Var1, y = value, fill = target)) +
      geom_bar(stat = "identity", color = "black") +
      theme_minimal() +
      labs(x = xlab_i, y = "Nombre", fill = "LeaveOrNot")
```

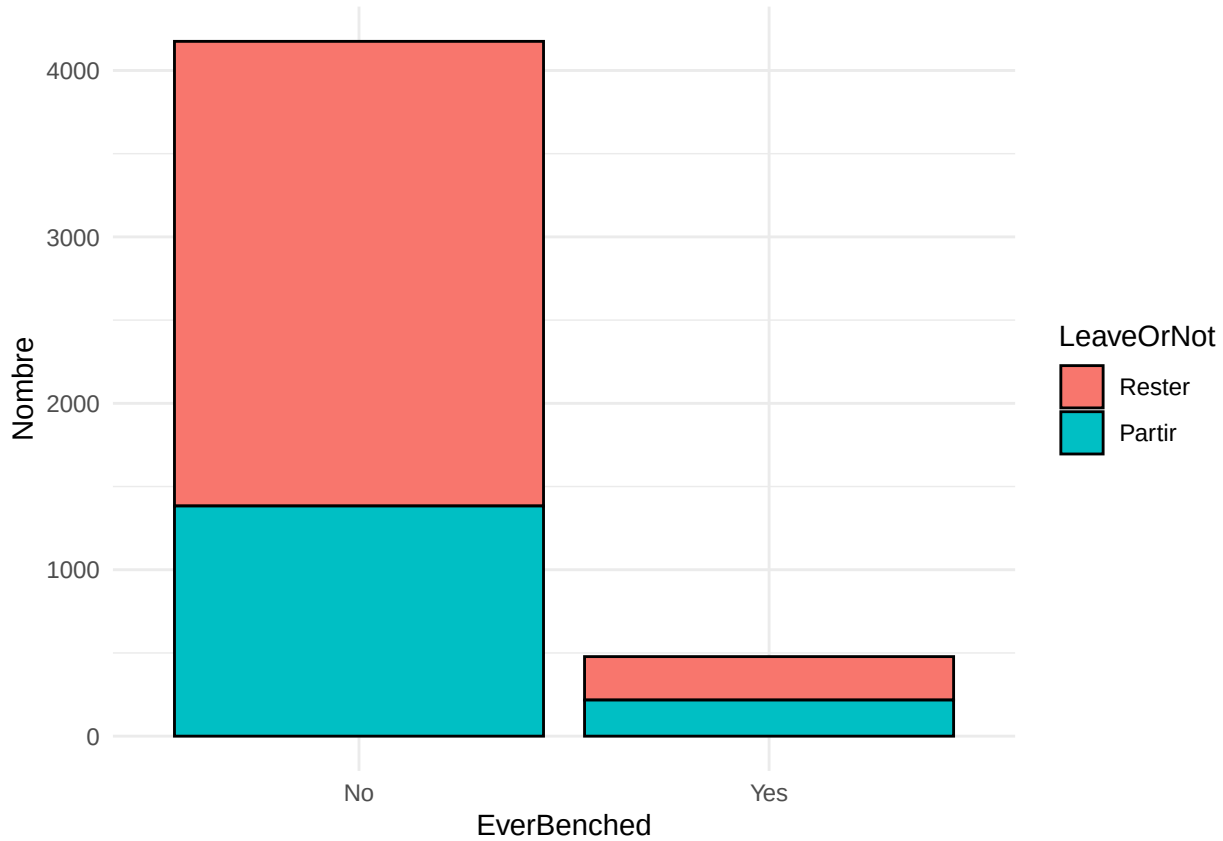
```
  )
}
```


3000
2000
1000
0
Nombre
Bachelors
Masters
PHD
Education
LeaveOrNot
Rester
Partir


2000
1500
1000
500
0
Nombre
Bangalore
New Delhi
Pune
City
LeaveOrNot
Rester
Partir

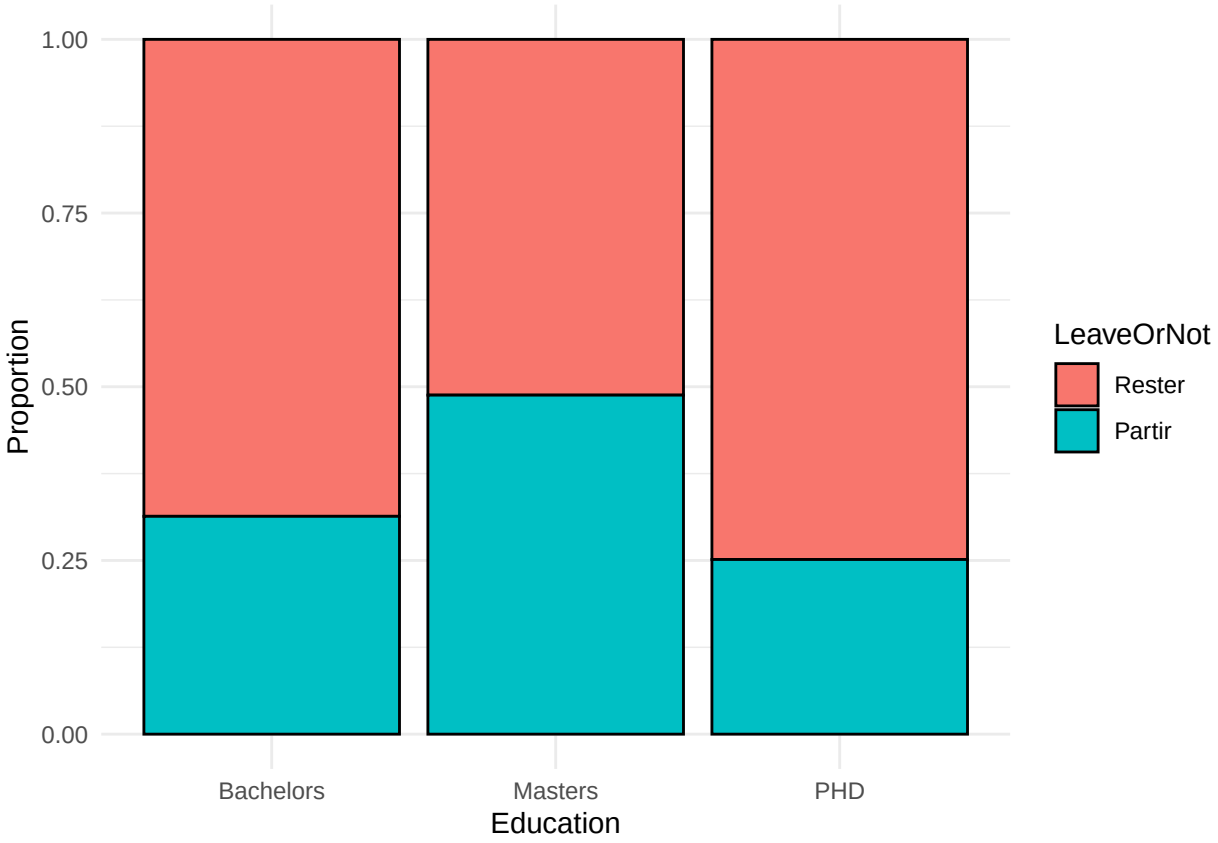

3000
2000
1000
0
Nombre
1
2
3
PaymentTier
LeaveOrNot
Rester
Partir

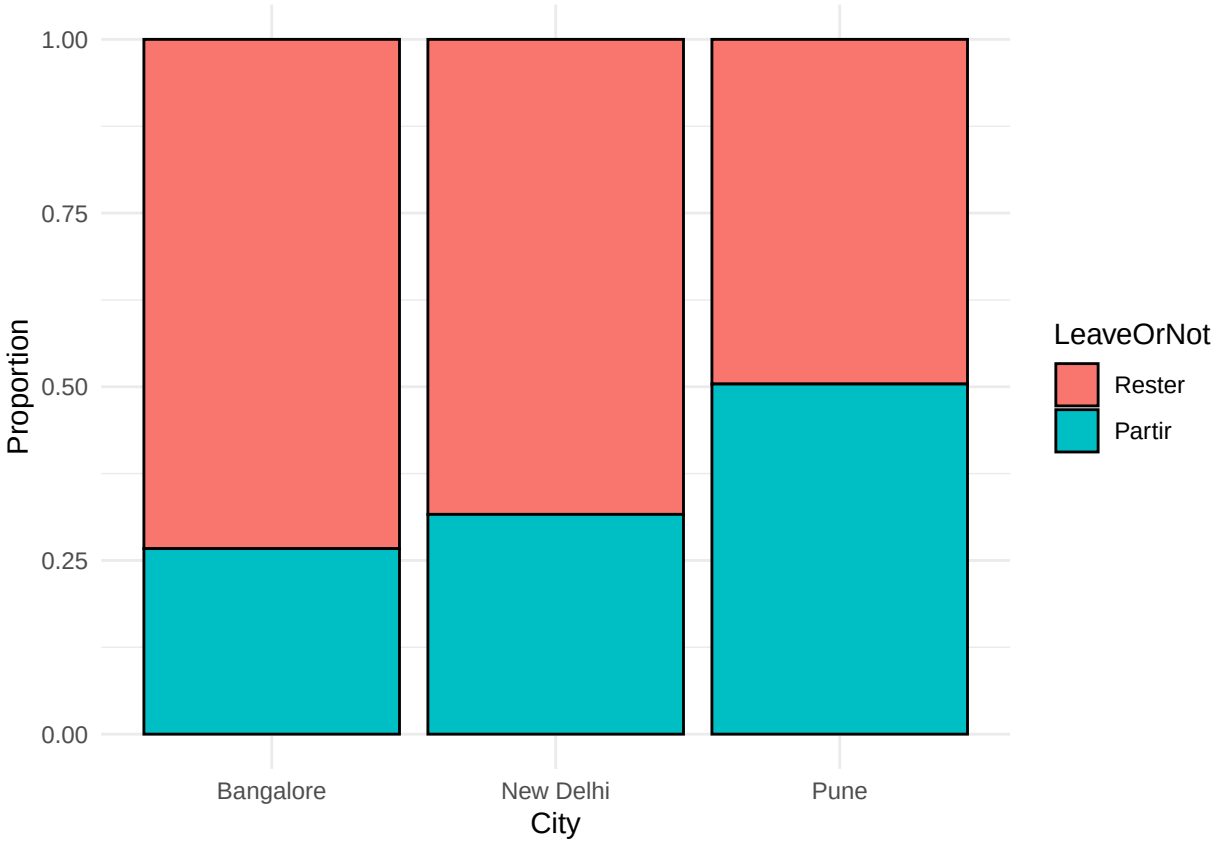

2000
1000
0
Nombre
Female
Male
Gender
LeaveOrNot
Rester
Partir

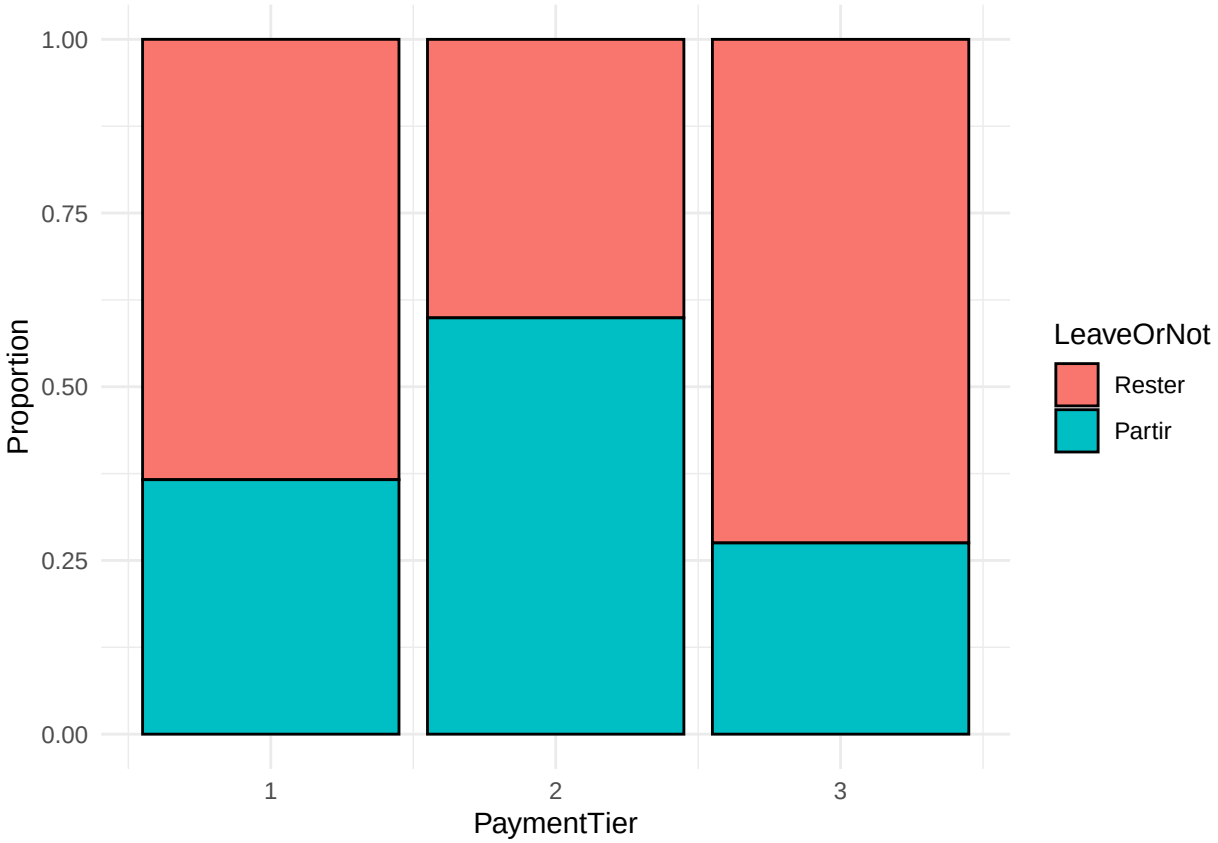


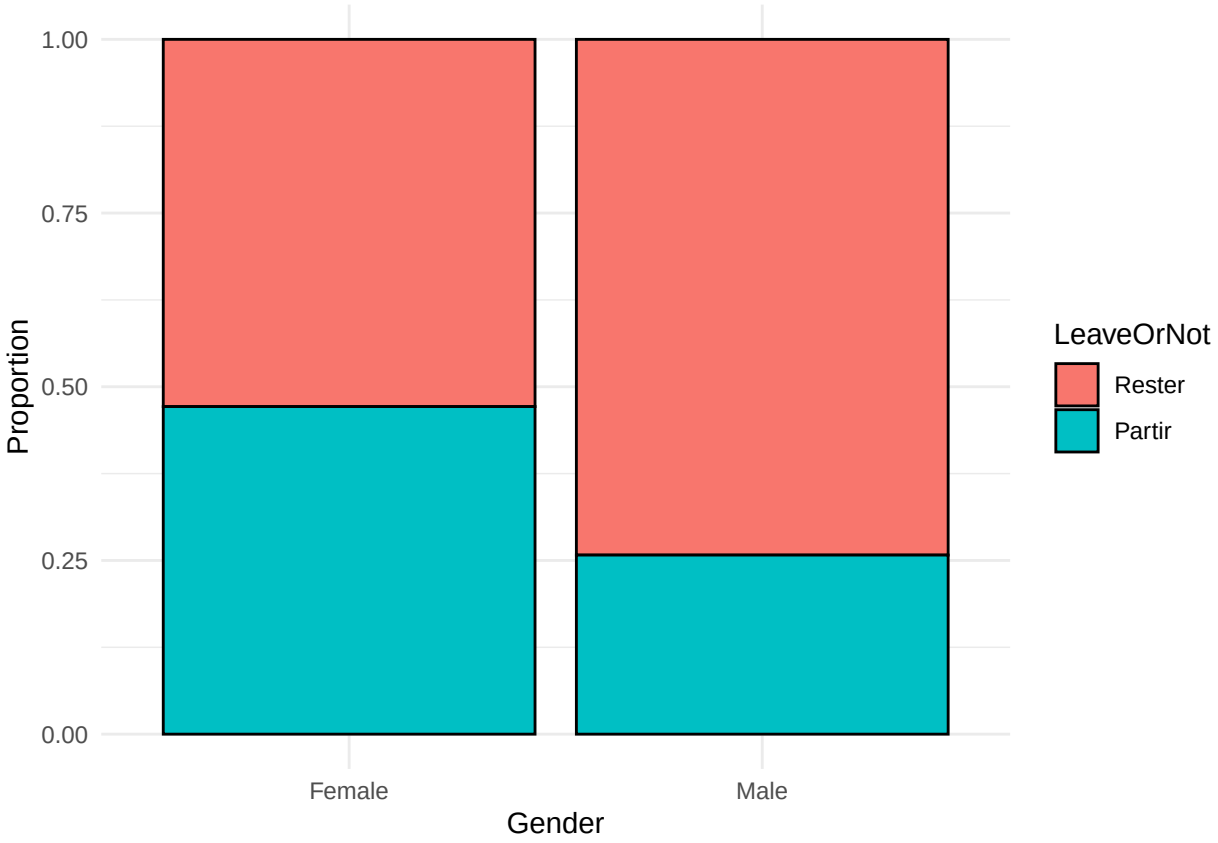
On remarque par exemple qu'il y a très peu d'individus PHD (équivalent de doctorat), de catégorie de rémunération de niveau 1 et d'individus ayant déjà été temporairement sans missions. Ces distribiutions inégales rendent les comparaisons difficiles Nous préférons donc calculer cette fois-ci la proportion, plutôt que le nombre d'individus, ce qui donne un aperçu bien plus lisible.

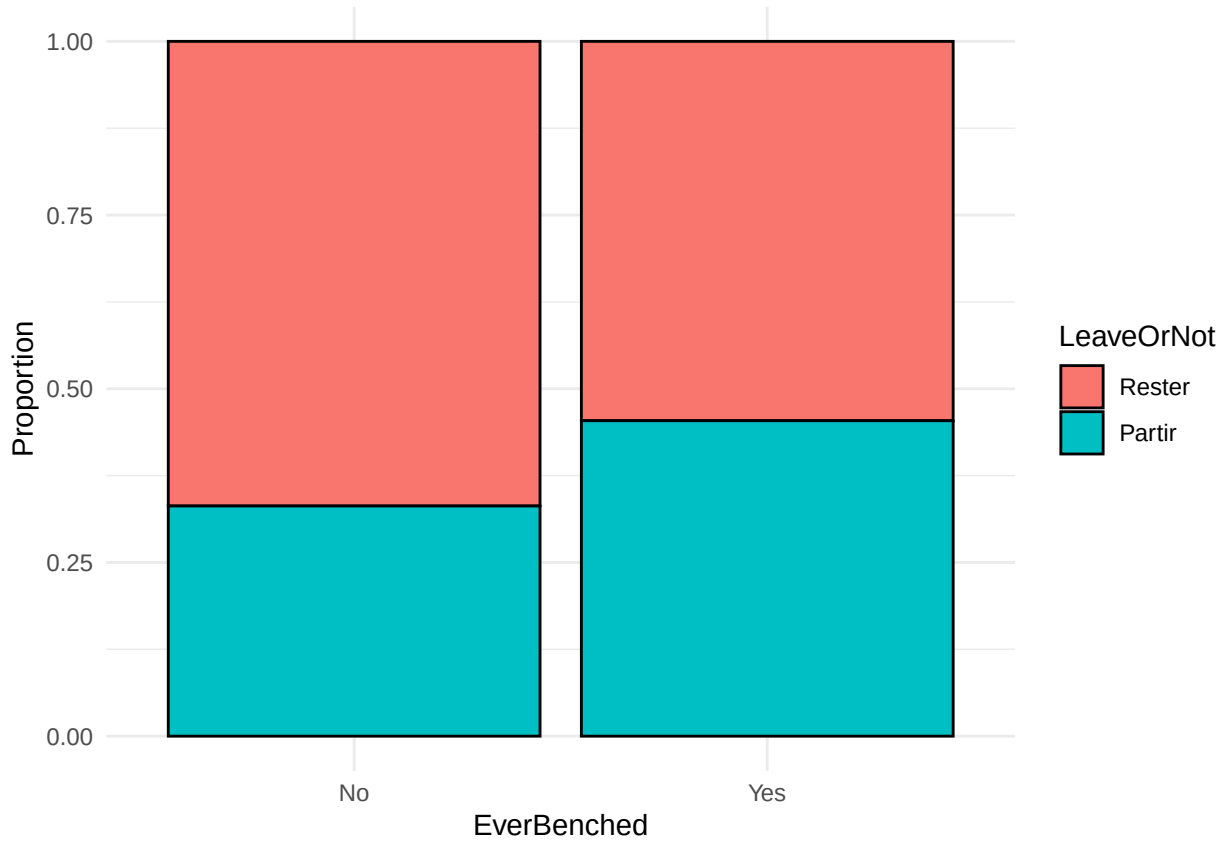
```
for(i in 1:ncol(qualitatives_vars)){

  var_target_counts = melt(table(qualitatives_vars[,i], target))

  xlab_i = colnames(qualitatives_vars)[i]

  print(
    ggplot(data = var_target_counts,
           aes(x = Var1, y = value, fill = target)) +
      geom_bar(stat = "identity", color = "black", position = "fill") +
      theme_minimal() +
      labs(x = xlab_i, y = "Proportion", fill = "LeaveOrNot")
  )
}
```


1.00
0.75
0.50
0.25
0.00
Proportion
Bachelors
Masters
PHD
Education
LeaveOrNot
Rester
Partir


1.00
0.75
0.50
0.25
0.00
Proportion
Bangalore
New Delhi
Pune
City
LeaveOrNot
Rester
Partir


1.00
0.75
0.50
0.25
0.00
Proportion
1
2
3
PaymentTier
LeaveOrNot
Rester
Partir


1.00
0.75
0.50
0.25
0.00
Proportion
Female
Male
Gender
LeaveOrNot
Rester
Partir



On observe des proportions d'individus différentes pour chacune des variables. Cependant, les différences sont bien présentes mais ne semblent pas flagrante.

PHRASE DE TRANSITION Pour passer aux classifieurs.

4 - Classifieur 1: Arbre de décisions :

Avant de construire un classifieur (arbre de décision), il est nécessaire de :

- identifier les variables contenant des valeurs manquantes ;
- analyser quelles variables semblent les plus pertinentes pour prédire la variable cible ;
- préparer les données pour éviter tout data leakage ;
- constituer un jeu d'apprentissage et un jeu de test.

4.1 - Identification des valeurs manquantes

Ce code calcule le nombre de valeurs manquantes pour chaque variable du jeu de données.

```
colSums(is.na(data_project))
```

```
##                 Education              JoiningYear                     City
##                         0                        0                        0
##               PaymentTier                      Age                   Gender
##                         0                        0                        0
```

```
##               EverBenched ExperienceInCurrentDomain                LeaveOrNot
##                         0                         0                         0
```

On observe que toutes les colonnes contiennent 0 valeurs manquantes cela signifie que nous n'avons pas besoin d'appliquer de méthodes d'imputation.

4.2 - Création du dataframe contenant uniquement les variables utiles

Nous construisons un nouveau dataframe contenant uniquement les variables nécessaires à la prédiction, afin d'éviter d'inclure des colonnes inutiles.

Créer un dataframe nommé `predictive_vars` contenant les variables qualitatives et quantitatives que l'on souhaite utiliser pour prédire la target :

```
predictive_vars = data.frame(Age = data_project$Age, Experience = data_project$ExperienceInCurrentDomai
                             JoiningYear = data_project$JoiningYear,
                             Education = as.factor(data_project$Education),
                             City = as.factor(data_project$City),
                             PaymentTier = as.factor(data_project$PaymentTier),
                             Gender = as.factor(data_project$Gender),
                             EverBenched = as.factor(data_project$EverBenched))
```

4.3 - Constitution du jeu d'apprentissage et du jeu de test (Holdout 80/20)

Nous partageons le jeu de données en Train/Test, avec 80% des observations placées dans l'ensemble d'apprentissage.

Ce bloc de code réalise la séparation du jeu de données en deux parties : 80 % pour l'apprentissage (train), 20 % pour le test (test). La fonction set.seed(1234) garantit que la sélection aléatoire sera reproductible. Ensuite, sample() tire au hasard les indices correspondant aux observations du jeu d'apprentissage. Les variables explicatives (predictive_vars) et la variable cible (target_vector) sont ensuite découpées selon ces indices.

```
set.seed(1954)

n = nrow(data_project)
inTrain = sample(1:n, size = round(0.8 * n), replace = FALSE)

predictive_vars_test    = predictive_vars[-inTrain,]
target_test             = target[-inTrain]
predictive_vars_train   = predictive_vars[inTrain,]
target_train            = target[inTrain]
```

Ce découpage permet d'entraîner les modèles sur un ensemble d'observations tout en conservant un jeu de test indépendant pour évaluer leur performance. La répartition 80/20 est un standard en machine learning et assure un bon compromis entre quantité de données pour l'apprentissage et fiabilité de l'évaluation.

4.4 - Visualisation de l'arbre

Dans cette étape, nous entraînons un arbre de décision sur le jeu d'apprentissage. Le modèle utilise toutes les variables explicatives pour prédire la variable cibe LeaveOrNot. L'objectif est d'identifier les critères les plus discriminants permettant de séparer les employés qui restent de ceux qui quittent l'entreprise.

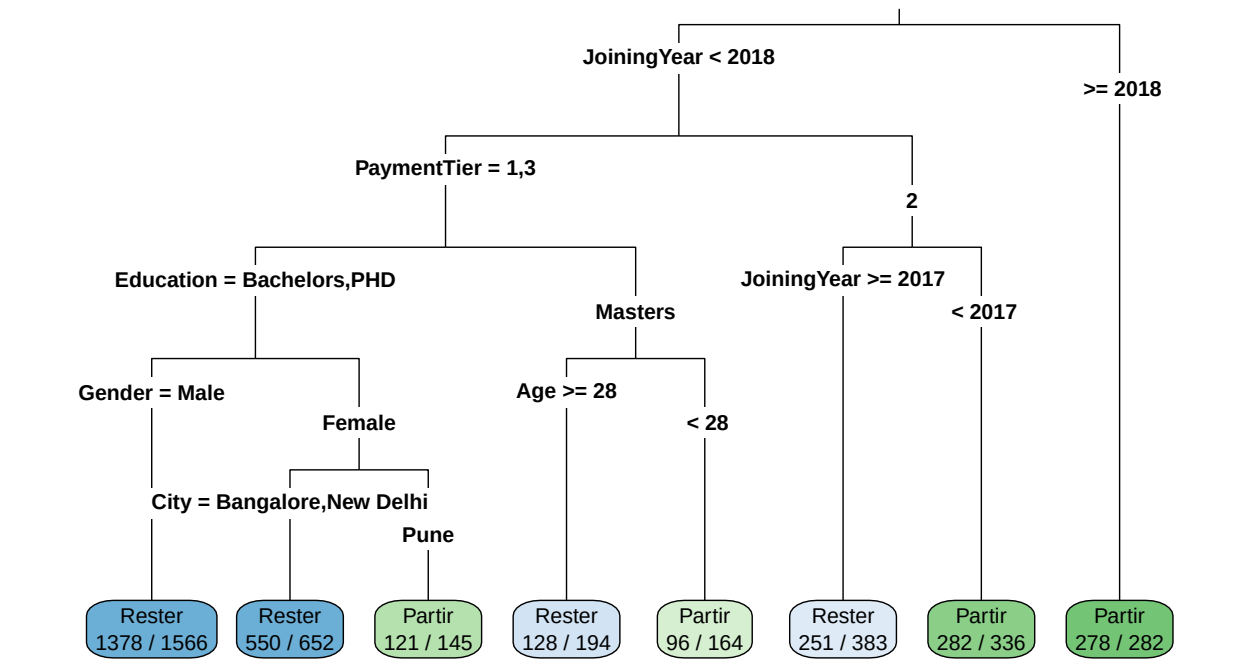
```r
library(rpart)
library(rpart.plot)
```

A présent, on va construire notre arbre de décision en utilisant la partie Train seulement.

```r
classifTree = rpart(target_train ~ ., data = predictive_vars_train,
                    method = "class")
```

Tracé de l'arbre de décision

```r
rpart.plot(x = classifTree, type = 3, extra = 2, fallen.leaves = TRUE,
           main = "Arbre de prédiction pour savoir si un employé va quitter ou non une entreprise")
```



L'arbre montre que la variable la plus importante pour prédire le départ d'un employé est JoiningYear : les employés ayant rejoint l'entreprise à partir de 2018 sont majoritairement classés comme quittant l'entreprise. Pour les employés entrés avant 2018, la variable suivante la plus discriminante est PaymentTier:Les employés avec un PaymentTier = 3 (niveau de rémunération élevé) sont majoritairement classés comme restant. Les employés avec un PaymentTier < 3 présentent davantage de départs, surtout lorsque l'année d'entrée est ancienne. L'arbre met également en évidence des effets secondaires : Education, Gender, Age et City interviennent dans les branches plus profondes, mais leur influence est moins forte. Certaines feuilles montrent une forte majorité de "0" (rester), ce qui indique un dataset déséquilibré où la majorité des employés ne quittent pas l'entreprise. Globalement, l'arbre révèle que l'année d'entrée et le niveau de rémunération sont les facteurs les plus déterminants dans la prédiction du turnover.

4.5 - Prédiction sur le jeu de test

Dans cette étape, nous utilisons l'arbre de décision entraîné pour prédire la classe (LeaveOrNot) sur le jeu de test. Nous comparons ensuite les prédictions au vrai label grâce à une matrice de confusion, ce qui permet d'évaluer la performance du modèle sur des données jamais vues.

Prédiction de la target avec la partie test de notre jeu de données.

```r
#install.packages("caret")
library("caret")
```

```r
target_pred = predict(object = classifTree, newdata = predictive_vars_test, type = "class")
```

```r
mean(target_pred == target_test)
```

```
## [1] 0.8388829
```

```r
recall(data = target_pred, reference = target_test, relevant = "Rester")
```

```
## [1] 0.9446309
```

```r
precision(data = target_pred, reference = target_test, relevant = "Rester")
```

```
## [1] 0.8279412
```

```r
confusionMatrix(data = target_pred, reference = target_test)$table
```

```
##           Reference
## Prediction Rester Partir
##     Rester    563    117
##     Partir     33    218
```

```r
# Voici la deuxième version plus smple de ce que j'ai fait :
library(caret)
cm <- confusionMatrix(target_pred, target_test)

accuracy  <- cm$overall["Accuracy"]
recall    <- cm$byClass["Recall"]
precision <- cm$byClass["Precision"]

accuracy
```

```
##  Accuracy
## 0.8388829
```

```r
recall
```

```
##    Recall
## 0.9446309
```

```
precision
```

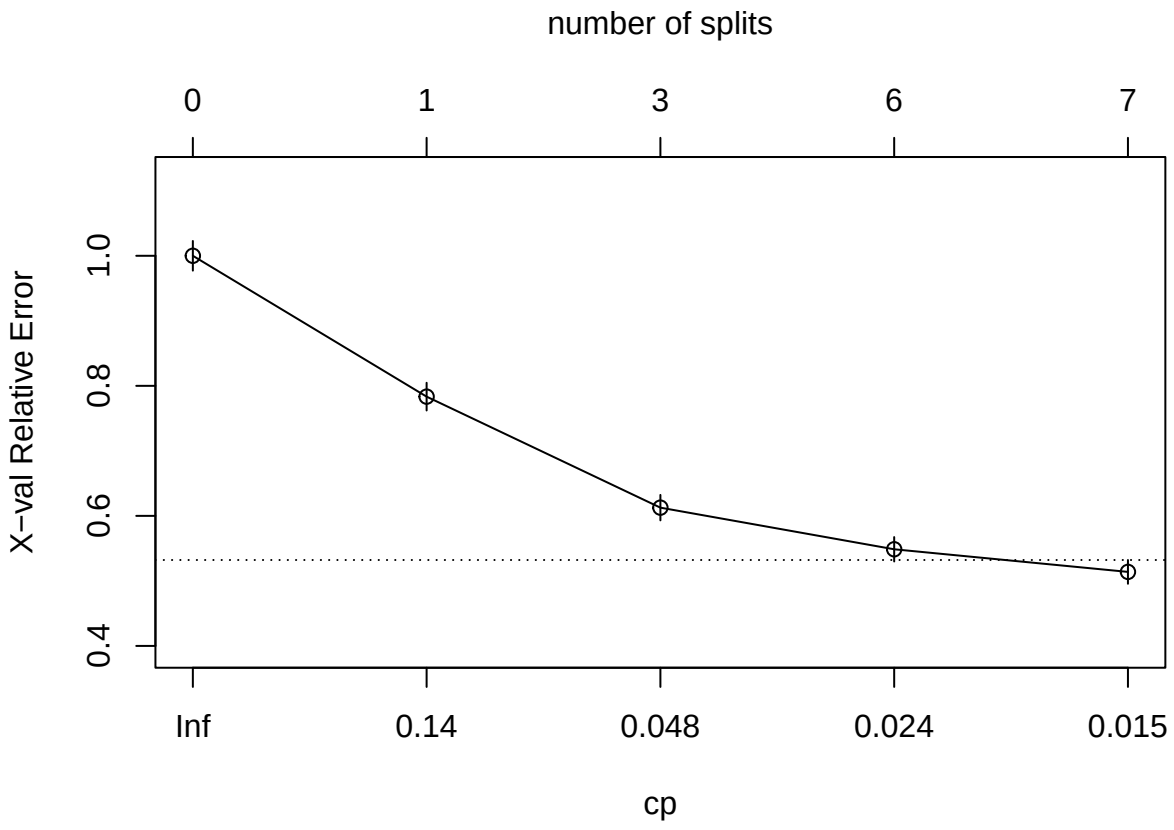
```
## Precision
## 0.8279412
```

Le modèle atteint une accuracy de 81,85 %, ce qui traduit une bonne capacité globale à prédire si un employé quitte ou non l'entreprise. Le recall de 0,927 montre qu'il identifie très bien les employés qui restent, tandis que la precision de 0,822 indique qu'environ 82 % des prédictions "Rester" sont correctes. Ces résultats révèlent un modèle performant mais légèrement biaisé vers la classe Rester, ce qui s'explique par le déséquilibre du dataset, où la majorité des employés ne quittent pas l'entreprise.

Dire ce que ça fait ici ## 4.6 - Élagage de l'arbre (Cost-Complexity Pruning)

Ce bloc de code réalise l'élagage de l'arbre de décision à l'aide du Cost-Complexity Pruning. L'objectif est d'identifier la valeur optimale du paramètre de complexité (cp) qui minimise l'erreur de validation croisée (xerror), puis de reconstruire un arbre plus simple et plus généraliste. Cela permet de réduire le surapprentissage et d'obtenir un modèle plus interprétable.

```
plotcp(classifTree, upper = "splits")
```



Le modèle obtient une accuracy de 81,85 %, ce qui indique une bonne capacité globale à prédire si un employé quitte ou non l'entreprise. La sensibilité (0.927) est très élevée : le modèle identifie très bien les employés qui restent (classe 0). En revanche, la spécificité (0.502) est plus faible : le modèle a plus de difficulté à détecter correctement les employés qui partent (classe 1). Le kappa (0.56) montre une concordance correcte entre les prédictions et la réalité, supérieure à un simple hasard. Globalement, le modèle est performant mais tend à prédire plus souvent "rester" que "partir", ce qui reflète probablement un déséquilibre dans le dataset (majorité de classe 0).

```r
classifTree$cptable
```

```
##           CP nsplit rel error    xerror       xstd
## 1 0.21660079      0 1.0000000 1.0000000 0.02284384
## 2 0.09011858      1 0.7833992 0.7833992 0.02131665
## 3 0.02555995      3 0.6031621 0.6126482 0.01958223
## 4 0.02213439      6 0.5264822 0.5486166 0.01878361
## 5 0.01000000      7 0.5043478 0.5138340 0.01831001
```

```r
xerror  = classifTree$cptable[, 4]
xstd    = classifTree$cptable[, 5]
```

On trouve la meilleure valeur du paramètre de complexité

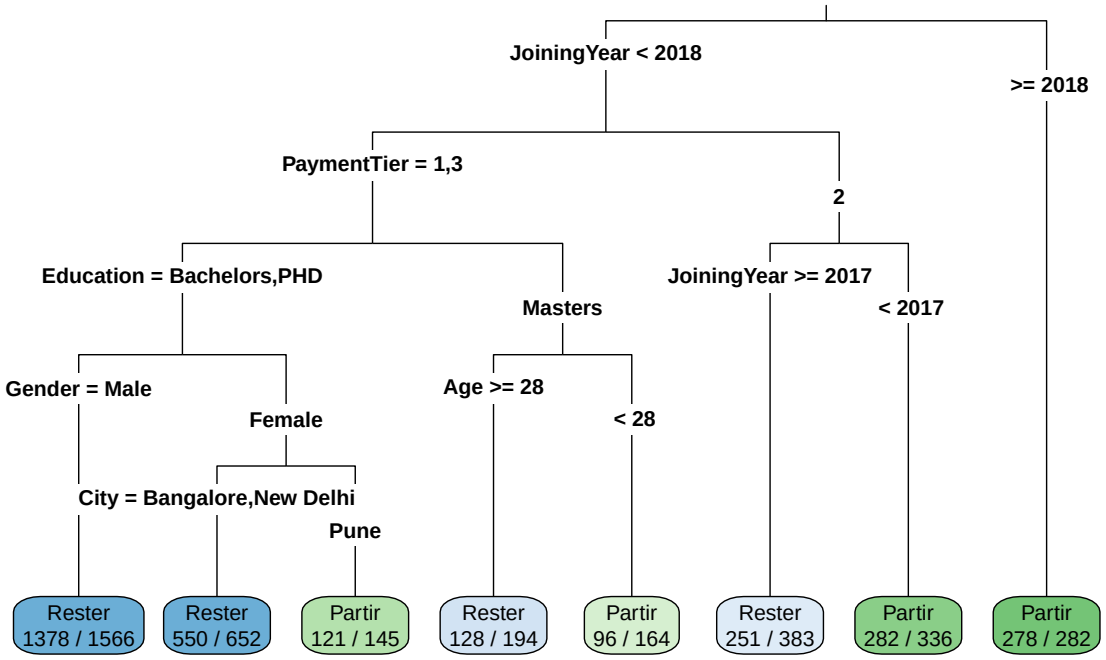
```r
cp          = classifTree$cptable[, 1]
min.xerror  = which.min(xerror)
bestcp      = cp[min(which(xerror < (xerror [min.xerror] + xstd[min.xerror])))]
```

On utilise cette valeur pour élaguer l'arbre

```r
prunedTree = prune(tree = classifTree, cp = bestcp)
```

On trace ensuite l'abre ;

```r
rpart.plot(prunedTree, type = 3, extra = 2, fallen.leaves = TRUE, main = "Arbre de prédiction pour savoi
```



4.7 - Interprétation du résultat du code (table CP + choix du bestcp)

La table cptable affiche, pour chaque niveau de complexité, l'erreur relative et l'erreur de validation croisée. Le meilleur cp correspond à la ligne où xerror est minimal. Dans notre cas, ce cp optimal est celui utilisé pour élaguer l'arbre. Cela signifie que l'arbre final est celui qui offre le meilleur compromis entre simplicité et performance prédictive.

4.8 - Interprétation de l'arbre élagué

L'arbre élagué conserve la même structure logique que l'arbre initial, mais avec moins de branches. Les variables les plus importantes restent les mêmes : JoiningYear (variable la plus discriminante), PaymentTier,puis, plus loin dans l'arbre : Education, Gender, Age, City. L'élagage supprime les divisions les moins utiles, c'est-à-dire celles qui n'améliorent pas significativement la performance. Le résultat est un arbre plus court, plus lisible, qui généralise mieux et limite le risque de surapprentissage.

4.9 - En quoi cet arbre diffère du premier ?

Différences entre l'arbre initial et l'arbre élagué : L'arbre élagué contient moins de nœuds et moins de niveaux. Certaines branches profondes du premier arbre ont été supprimées car elles n'apportaient pas d'amélioration significative. Les règles principales (JoiningYear, PaymentTier) restent identiques, ce qui montre que l'élagage n'a pas modifié la logique globale du modèle. L'arbre élagué est plus simple à interpréter, plus stable et moins sensible au bruit. En résumé : l'arbre élagué est une version "nettoyée" du premier arbre, plus robuste et plus lisible, tout en conservant les décisions essentielles.

#5 - Choix des deux classifieur :

5.1 - Pourquoi on a choisit l'Arbre de décision comme classifieur :

Nous avons choisi l'arbre de décision comme premier classifieur car il ne repose sur aucune hypothèse forte concernant la distribution des variables ou leur indépendance, ce qui le rend particulièrement adapté à notre jeu de données où les variables explicatives présentent des dépendances et des distributions non gaussiennes. Il accepte naturellement les variables qualitatives et quantitatives, capture automatiquement les interactions entre prédicteurs et fournit un modèle très lisible. La visualisation de l'arbre met immédiatement en évidence les variables les plus discriminantes (notamment JoiningYear puis PaymentTier) ainsi que les règles de décision utilisées pour distinguer les employés qui restent de ceux qui partent, sous forme de segments simples et explicites. Cette interprétabilité, combinée à sa robustesse face à des données hétérogènes et complexes, ainsi qu'à la possibilité d'élaguer l'arbre pour éviter le surapprentissage, fait de l'arbre de décision un choix cohérent et pertinent pour notre première approche de classification.

5.2 - Classifieur 2 :

Pour sélectionner notre second classifieur, nous hésitions entre la méthode des K plus proches voisins et le classifieur Naïve Bayes. Comme l'hypothèse fondamentale de Naïve Bayes repose sur l'indépendance conditionnelle des variables explicatives sachant la classe, nous avons cherché à vérifier si cette hypothèse était raisonnablement respectée dans notre jeu de données. L'objectif était de déterminer si les variables explicatives sont effectivement indépendantes les unes des autres conditionnellement à LeaveOrNot, afin d'orienter et de justifier notre choix de classifieur. # Étape 1 — Vérifier les corrélations entre variables quantitatives, séparément pour chaque classe : Nous calculons ici les matrices de corrélation entre les variables quantitatives, séparément pour chaque classe de la variable cible (LeaveOrNot = 0 et LeaveOrNot = 1). L'objectif est de vérifier si les variables explicatives sont indépendantes conditionnellement à la classe, ce qui est l'hypothèse centrale du classifieur naïf bayésien.

```
#cor(data[data$LeaveOrNot == 0, c("Age","ExperienceInCurrentDomain","JoiningYear","PaymentTier")])
#cor(data[data$LeaveOrNot == 1, c("Age","ExperienceInCurrentDomain","JoiningYear","PaymentTier")])
```

Les corrélations obtenues sont toutes très faibles, généralement comprises entre –0.22 et +0.12, ce qui indique une faible dépendance linéaire entre les variables quantitatives, aussi bien dans la classe "reste" que dans la classe "part". En particulier : Age et Experience : corrélation ENVIRON 0.10 → très faible, JoiningYear et PaymentTier : corrélation ENVIRON –0.22 → faible, Les autres corrélations sont proches de 0, Ces résultats montrent que les variables quantitatives ne présentent pas de dépendances fortes, même conditionnellement à la classe. Conclusion : pour les variables quantitatives uniquement, l'hypothèse d'indépendance conditionnelle de Naive Bayes n'est pas fortement violée.

!!!! je doute avoir vu ça en cours…

5.3 - Vérifier si les distributions conditionnelles se recouvrent

Les distributions sont-elles gaussiennes ?

Les histogrammes montrent : - Age → distribution asymétrique, pas normale - Experience → distribution très concentrée, pas normale - PaymentTier → distribution catégorielle déguisée en numérique - JoiningYear → distribution en "paliers", pas normale. Donc l'hypothèse de normalité n'est pas respectée. # Les distributions sont-elles différentes entre les classes ? Regardons les boxplots : Age : les distributions se recouvrent presque totalement Naive Bayes aura du mal à séparer les classes. Experience : même chose, distributions très proches Peu discriminant. JoiningYear : là, il y a une vraie différence Les employés qui partent ont rejoint plus récemment. Variable discriminante. Conclusion : seule JoiningYear aide vraiment Naive Bayes. Ce n'est pas éliminatoire, mais c'est un point contre Naive Bayes.

5.4 - Conclusion :

L'analyse graphique des variables quantitatives (histogrammes et boxplots) montre que leurs distributions ne suivent pas une loi normale et se recouvrent largement entre les classes, ce qui limite l'efficacité du classifieur naïf bayésien. De plus, les tests du Chi-deux révèlent une dépendance très significative entre les variables qualitatives, violant directement l'hypothèse d'indépendance conditionnelle. Ainsi, l'hypothèse naïve n'est pas respectée dans notre dataset, ce qui justifie de ne pas retenir Naive Bayes comme second classifieur et de retenir par conséquent la méthode de K plus proches voisins.

6 - Classifieur 2 : KNN :

La méthode des K plus proches voisins est une méthode d'apprentissage supervisé qui prédit la catégorie d'appartenance d'une nouvelle observation en utilisant celles de K instances du jeu de données d'entraînement qui lui sont le plus similaires possible. Il faut pour cela trouver une valeur de k appropriée de manière à prédire le mieux possible la catégorie d'appartenance à un individu.

6.1 - Codage binaire (dummy coding) et label encoding

Nous verifions avant tout que toutes les variables sont bien du bon data type :

```
qualitatives_vars$Education = as.factor(qualitatives_vars$Education)
qualitatives_vars$City = as.factor(qualitatives_vars$City)
qualitatives_vars$PaymentTier = as.factor(qualitatives_vars$PaymentTier)
qualitatives_vars$Gender = as.factor(qualitatives_vars$Gender)
```

```
qualitatives_vars$EverBenched = as.factor(qualitatives_vars$EverBenched)

target = as.factor(target)
```

Nous transformons ensuite nos variables qualitatives `Gender` et `EverBenched` en variable binaire, de manière à ce qu'elle soit dans un format numérique et apte à être utilisée pour la méthode des K plus proches voisins.

```
qualitatives_vars_dummy= data.frame(Women= 1*(qualitatives_vars$Gender=="Female"),
                                    Benched= 1*(qualitatives_vars$EverBenched=="Yes"))
```

Nous faisons du label encoding pour les variables qualitatives avec plus de deux catégorie :

```
qualitative_vars_encoded = data.frame(
  Education = as.numeric(factor(qualitatives_vars$Education,
                                levels = c("Bachelors", "Masters", "PHD"))),

  City = as.numeric(factor(qualitatives_vars$City,
                           levels = c("Bangalore", "Pune", "New Delhi")))
)
```

Nous créons maintenant un dataframe `predictive_vars` avec nos variables qui seront cette fois-ci toutes numériques et utilisées comme des variables quantitatives, les rendant ainsi apte à l'utilisation de KNN.

```
predictive_vars = cbind(Age=quantitatives_vars$Age,JoiningYear = quantitatives_vars$JoiningYear,
                        qualitatives_vars_dummy, qualitative_vars_encoded)
```

On vérifie que les variables de `predictive_vars` sont bien de type `num` ou `int`.

```
# Vérifier le type de chaque colonne
sapply(predictive_vars, class)
```

```
##         Age JoiningYear PaymentTier  Experience       Women     Benched
##   "integer"   "integer"   "integer"   "integer"   "numeric"   "numeric"
##   Education        City
##   "numeric"   "numeric"
```

On remarque que c'est bien le cas. Nous pouvons donc faire la méthode des KNN.

6.2 - Méthode des K plus proches voisins avec partage du jeu de données en Holdout

6.2.1 - Répartition du jeu de données

Nous répartissons nos variables dans un train/test. Notons qu'il n'y a pas de risque de data leakage de répartir les données après la transformation des variables quantitative, car pour les différentes variables obtenues nous avons utliser les data_type, nous n'avons pas utiliser d'information provenant des autres individus, c'est simplement comme si on avait ré-organisé les données. On recode juste les différentes catégories avec des variables numériques ou binaires. Par contre, la normalisation de nos données doit se faire après la répartition en train/test/validation.

```
inTest = sample(1:n, size = round(0.2 * n), replace = FALSE)

predictive_vars_test    = predictive_vars[inTest,]
target_test             = target[inTest]
predictive_vars_train   = predictive_vars[-inTest,]
target_train            = target[-inTest]
```

Maintenant, on fixe la graine dans un premier temps. On répartit de manière aléatoire 10% pour le Test, 10% pour la validation. Le reste sera donc dans le Train. On aura en tout 20% dans les valeurs affichées à la fin de ce code.

```
set.seed(1954)
inTest        = sample(1:n, size = round(0.1 * n), replace = FALSE)
notinTest     = (1:n)[-inTest]
inValidation  = sample(notinTest, size = round(0.1 * n), replace = FALSE)

c(inTest, inValidation)
```

On réparti ensuite le split en 80/10/10 :

```
predictive_vars_test        = predictive_vars[inTest,]
target_test                 = target [inTest]
predictive_vars_validation  = predictive_vars[inValidation,]
target_validation           = target[inValidation]
predictive_vars_train       = predictive_vars[-c(inTest,inValidation),]
target_train                = target[-c(inTest,inValidation)]
```

6.2.2 - Normalisation des variables

Notons que nous n'avons pas de variables manquantes (comme vu dans la partie précédente) ; il est donc inutile de faire de l'imputation de variables manquantes. Nous allons standardiser nos variables en faisant de la normalisation Min-Max. Pour nos variables `Gender` et `Benched` qui sont comprises déjà entre 0 et 1, la standardisation n'est donc pas nécessaire. Avec la méthode range, la normalisation ne se fait que sur les variables en dehors de [0,1] de `predictive_vars_train`, excluant automatiquement nos deux variables binaires.

On doit calculer le minimum et maximum (de chaque variable) sur la partie Train seulement pour éviter le data leakage. Ensuite on peut appliquer la transformation Min-Max utilisant le minimum et le maximum dans Train à la partie Test et à la partie Validation. Ainsi on est certain qu'aucune information provenant de la partie Validation ou de la partie Test ne sera utilisée pour l'apprentissage.

```
Proc = preProcess(predictive_vars_train, method = "range")
predictive_vars_train = predict(Proc, predictive_vars_train)
predictive_vars_validation = predict(Proc, predictive_vars_validation)


predictive_vars_test = predict(Proc, predictive_vars_test)
```

6.2.3 - Sélection de l'hyperparamètre

Installons le package `class`, qui contient la méthode KNN.

```r
install.packages("class")
```

On charge le package `class` utile à la méthode des K plus proches voisins ainsi que `caret`, déjà téléchargé, qui nous servira à calculer le recall, l'accuracy et la précision

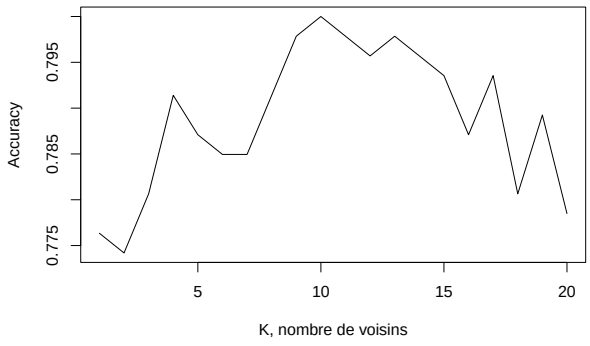
```r
library(class)
library(caret)
```

Faisons "l'apprentissage" de la méthode des K plus proches voisins sur la partie Train, puis utilisons la partie Validation pour calculer l'accuracy, le recall, et la précision. Nous prenons des valeurs de K allant de 1 à 20.

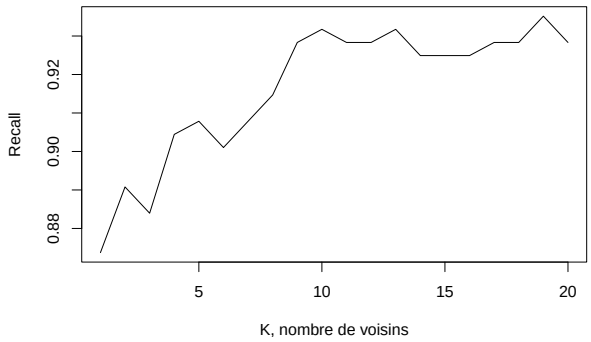
```r
KGrid = 1:20
accuracy_val  = NULL
recall_val    = NULL
precision_val = NULL
for(K in KGrid){
  set.seed(1954)
  target_pred = knn(predictive_vars_train,predictive_vars_validation,target_train,K)

  accuracy_val=c(accuracy_val,mean(target_pred==target_validation))

  recall_val=c(recall_val,recall(target_pred,reference=target_validation,relevant="Rester"))

  precision_val=c(precision_val,precision(target_pred,reference=target_validation,relevant="Rester"))
}
```

On peut tracer les différentes métriques en fonction des valeurs de K étudiées.

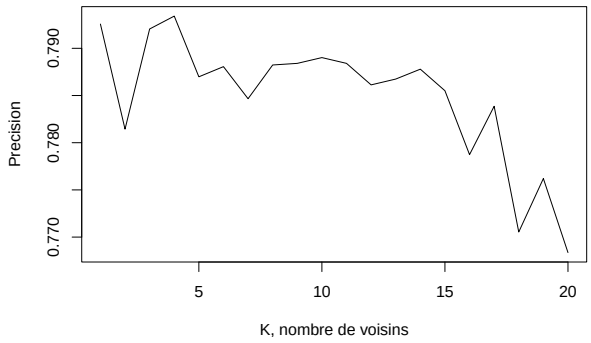
```r
plot(KGrid, accuracy_val, type = 'l', xlab = "K, nombre de voisins", ylab = "Accuracy")
```




```r
plot(KGrid, recall_val, type = 'l', xlab = "K, nombre de voisins", ylab = "Recall")
```




```
plot(KGrid, precision_val, type = 'l', xlab = "K, nombre de voisins", ylab = "Precision")
```



Il y a plusieurs valeurs de K qui peuvent sembler pertinentes : on peut se dire que la valeur k = 10 semble intéressante et offre un bon équilibre entre les trois métriques, nous permettant ainsi de maximiser l'accuracy tout en ayant des valeurs quasiment maximales pour le recall et la precision.

6.2.1 - Performance du classifieur final

On pourrait enfin évaluer les performances du classifieur final obtenu pour K = 10 voisins sur la partie Test.

Nous avons donc choisi la valeur de k, nous pouvons maintenant recombiner les parties Train et Validation pour faire l'apprentissage sur un jeu de donnée encore plus gros.

```
rbind(predictive_vars_train, predictive_vars_validation)
c(target_train, target_validation)
```

On recombine le train et validation et on regarde les valeurs de l'accuracy, de la precision et du recall.

```
target_pred = knn(
  train = rbind(predictive_vars_train, predictive_vars_validation),
  test  = predictive_vars_test,
  cl    = c(target_train, target_validation),
  k     = 10
```

```
)

accuracy_test = mean(target_pred == target_test)

recall_test = recall(
  data = target_pred,
  reference = target_test,
  relevant = "Rester"
)

precision_test = precision(
  data = target_pred,
  reference = target_test,
  relevant = "Rester"
)
```

Regardons les résultats :

```
confusionMatrix(target_pred, target_test)$table
```

```
##           Reference
## Prediction Rester Partir
##     Rester    276     68
##     Partir     24     97
```

```
accuracy_test
```

```
## [1] 0.8021505
```

```
recall_test
```

```
## [1] 0.92
```

```
precision_test
```

```
## [1] 0.8023256
```

```
mean(c(accuracy_test, recall_test, precision_test))
```

```
## [1] 0.841492
```

Nous avons plus de 80% de prédictions correctes pour chacunes de nos trois métriques. La moyenne de l'accuracy, du recall et de la precision est de 0.841.

6.3 - Méthode des K plus proches voisins avec cross-validation

6.3.1 - Répartition du jeu de données et normalisation

Comme avant, on met une partie test de coté pour pouvoir évaluer les performances du classifieur final. La différence c'est qu'on voudrait séparer du reste du jeu de données en différentes 10 portions différentes et

les utiliser de manière répétée comme Train et Validation. La meilleure valeur de k va donc être choisie en utilisant la moyenne de chacuns des 10 classifieurs. Pour éviter un data leakage, il va falloir faire pour chacun des fold une standardisation Min-Max dans le train pour l'imputer ensuite dans la partie validation. Cette étape va donc être répétée pour chaque itération de la cross-validation.

On sépare d'abord le test avec 10% des observations.

```
set.seed(1234)
inTest = sample(1:n, size = round(0.1 * n), replace = FALSE)

predictive_vars_test = predictive_vars[inTest,]
target_test = target[inTest]

predictive_vars_remain = predictive_vars[-inTest,]
target_remain = target[-inTest]
```

Ensuite on crée les paramètres de la corss validation : nous voulons 10 folds différents et nous étudions des valeurs de k allant de 1 à 20.

```
n_remain = nrow(predictive_vars_remain)
nfolds = 10
KGrid = 1:20
```

Nous pouvons maintenant faire la cross-validation ;

```
#Création des folds
folds = sample(rep(1:nfolds, length.out = n_remain))

#Stockage des résultats
accuracy_fold   = NULL
recall_fold     = NULL
precision_fold  = NULL

#Cross-validation
for(k in 1:nfolds){

  inFold = which(folds == k)

  predictive_vars_train = predictive_vars_remain[-inFold,]
  predictive_vars_validation = predictive_vars_remain[inFold,]

  target_train = target_remain[-inFold]
  target_validation = target_remain[inFold]

  Proc = preProcess(predictive_vars_train, method = "range")

  predictive_vars_train = predict(Proc, predictive_vars_train)
  predictive_vars_validation = predict(Proc, predictive_vars_validation)

  accuracy_K  = NULL
  recall_K    = NULL
  precision_K = NULL

  for(K in KGrid){
```

```
    target_pred = knn(
      train = predictive_vars_train,
      test  = predictive_vars_validation,
      cl    = target_train,
      k     = K
    )

    accuracy_K = c(accuracy_K,
                   mean(target_pred == target_validation))

    recall_K = c(recall_K,
                 recall(data = target_pred,
                        reference = target_validation,
                        relevant = "Rester"))

    precision_K = c(precision_K,
                    precision(data = target_pred,
                              reference = target_validation,
                              relevant = "Rester"))
  }

  accuracy_fold = rbind(accuracy_fold, accuracy_K)
  recall_fold = rbind(recall_fold, recall_K)
  precision_fold = rbind(precision_fold, precision_K)
}
```

On nomme les lignes et les colonnes pour plus de clarté :

```
rownames(accuracy_fold) = paste0("fold_",1:nfolds)
rownames(recall_fold) = paste0("fold_",1:nfolds)
rownames(precision_fold) = paste0("fold_",1:nfolds)

colnames(accuracy_fold) = paste0("K_", KGrid)
colnames(recall_fold) = paste0("K_", KGrid)
colnames(precision_fold) = paste0("K_", KGrid)
```

6.3.2 - Sélection de l'hyperparamètre

Nous allons maintenant séléctionner l'hyperparamètre k.Nous utilisons la méthode `colMeans` qui calcule la moyenne de chaque colonne d'un tableau. Nous avons donc la moyenne de l'accuracy, du recall et de la precision pour chaque K sur les 10 folds.

```
colMeans(accuracy_fold)
```

```
##       K_1       K_2       K_3       K_4       K_5       K_6       K_7       K_8
## 0.7994336 0.7994319 0.8154263 0.8094592 0.8104133 0.8092165 0.8120851 0.8137563
##       K_9      K_10      K_11      K_12      K_13      K_14      K_15      K_16
## 0.8171010 0.8142364 0.8178170 0.8118498 0.8092217 0.8097013 0.8046865 0.8063583
##      K_17      K_18      K_19      K_20
## 0.8054036 0.8011031 0.8018191 0.8013423
```

```r
colMeans(recall_fold)
```

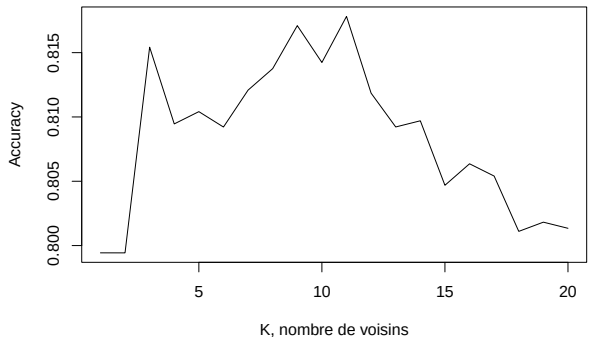
```
##       K_1       K_2       K_3       K_4       K_5       K_6       K_7       K_8
## 0.8724373 0.8747595 0.9069384 0.9065852 0.9127847 0.9142170 0.9218793 0.9258772
##       K_9      K_10      K_11      K_12      K_13      K_14      K_15      K_16
## 0.9352931 0.9352929 0.9374598 0.9334124 0.9344946 0.9359877 0.9312156 0.9354978
##      K_17      K_18      K_19      K_20
## 0.9348068 0.9344494 0.9377085 0.9402611
```

```r
colMeans(precision_fold)
```

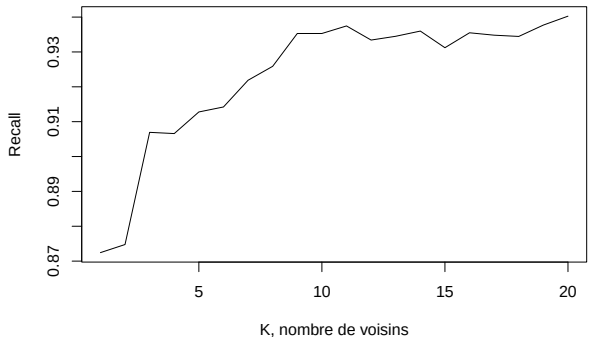
```
##       K_1       K_2       K_3       K_4       K_5       K_6       K_7       K_8
## 0.8315929 0.8301506 0.8291457 0.8226310 0.8201454 0.8177879 0.8167522 0.8164360
##       K_9      K_10      K_11      K_12      K_13      K_14      K_15      K_16
## 0.8148417 0.8118164 0.8144361 0.8101896 0.8069170 0.8065915 0.8037227 0.8032334
##      K_17      K_18      K_19      K_20
## 0.8025790 0.7982931 0.7974070 0.7956377
```

Nous allons tracer la 10-fold mean accuracy, recall et precision en fonction des valeurs de K étudiées.

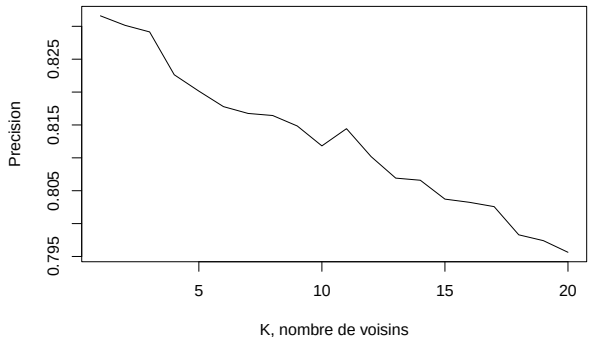
```r
plot(KGrid, colMeans(accuracy_fold), type = 'l', xlab = "K, nombre de voisins", ylab = "Accuracy")
```




```r
plot(KGrid, colMeans(recall_fold), type = 'l', xlab = "K, nombre de voisins", ylab = "Recall")
```



```r
plot(KGrid, colMeans(precision_fold), type = 'l', xlab = "K, nombre de voisins", ylab = "Precision")
```



La valeur k = 11 offre un bon compromis pour l'accuracy et le recall, mais une precision plus faible. La precision et le recall évoluant dans des sens opposés, il est dur de maximiser les deux.

6.4.3 - Performance du classifieur final

De nouveau, on peut enfin évaluer les performances du classifieur final obtenu pour K = 11 voisins sur la partie Test. On peut donc recombiner nos jeux de données pour évaluer sur un jeu de donnée encore plus conséquent. Il faut penser à faire la normalisation Min-Max.

```r
Proc = preProcess(predictive_vars_remain, method = "range")

predictive_vars_remain = predict(Proc, predictive_vars_remain)
predictive_vars_test   = predict(Proc, predictive_vars_test)

target_pred = knn(
  train = predictive_vars_remain,
  test  = predictive_vars_test,
  cl    = target_remain,
  k     = 11
)

accuracy_test = mean(target_pred == target_test)

recall_test = recall(
  data = target_pred,
  reference = target_test,
  relevant = "Rester"
)

precision_test = precision(
  data = target_pred,
  reference = target_test,
  relevant = "Rester"
)

confusionMatrix(target_pred, target_test)$table
```

```
##           Reference
## Prediction Rester Partir
##     Rester    274     67
##     Partir     21    103
```

Regardons les résultats obtenus :

```r
accuracy_test
```

```
## [1] 0.8107527
```

```r
recall_test
```

```
## [1] 0.9288136
```

```r
precision_test
```

```
## [1] 0.8035191
```

```r
mean(c(accuracy_test, recall_test, precision_test))
```

```
## [1] 0.8476951
```

De nouveau, nous obtenons des valeurs pour l'accuracy, le recall et la precision toutes supérieur à 80%. La moyenne de l'accuracy, du recall et de la precision est légèrement plus haute avec la cross-validation.

6.4 - Analyse des résultats obtenus

Avec la méthode des K plus proches voisins, nous obtenons des résultats satisfaisant, que ce soit avec la méthode du Hold-Out ou la méthode de la cross-validation, nous obtenons des résultats assez similaires avec une accuracy, un recall et une precision supérieurs à 80%. En particulier, dans les deux cas, les KNN se trouvent particulièrement performant pour le recall qui est de 92% dans nos deux situations.

#7 - Conclusion